STAR RX72N - motors_rtos Bench Test

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Directory Hierarchy

1.1 Directories

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

Chapter 3

Directory Documentation

3.1 motors_rtos Directory Reference

Files

- file clock.c

  RX72N clock init for motors_rtos: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
- file cmt0.c

  CMT0 ThreadX tick source for the PLL-clocked motors_rtos test.
- file linker.ld
- file main.c

  motors_rtos – motors + encoders under Azure RTOS ThreadX.
- file sci9.c
- file stubs.c

  No-op log/UART stubs for the motor spin test.

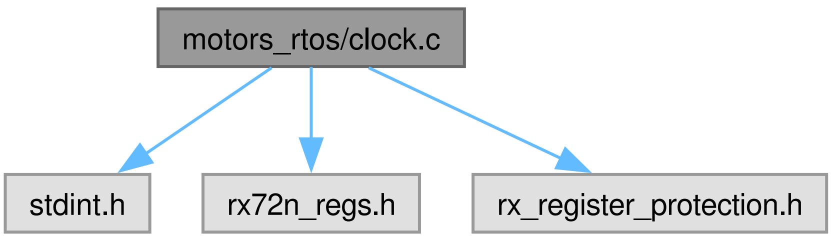
Chapter 4

File Documentation

4.1 motors_rtos/clock.c File Reference

RX72N clock init for motors_rtos: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.

#include <stdint.h>
#include ”rx72n_regs.h”
#include ”rx_register_protection.h”
Include dependency graph for clock.c:



Enumerations

- enum clock_extra_addrs_t : uintptr_t { k_packcr_addr = 0x00080044U }
- enum clock_word_constants_t : uint16_t { k_pllcr_mosc_x10 = 0x1300U }
- enum clock_sckcr_t : uint32_t { k_sckcr_value = 0x21C21211U }
- enum clock_byte_constants_t : uint8_t {
  k_sub_clock_stopped = 0x01U , k_main_osc_enabled = 0x00U , k_pll_enabled = 0x00U , k_oscovfsr_moovf = 0x01U ,
  k_oscovfsr_plovf = 0x04U }
- enum clock_timeout_t : uint32_t { k_main_osc_poll_max = 500000U , k_pll_lock_poll_max = 500000U }

Functions

- void clock_init (void)

4.1.1 Detailed Description

RX72N clock init for motors_rtos: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.

Self-contained reduced version of src/rx_clock_power_init.c, stripped of the logging, asserts, simulator branches, and PPLL/USB clock paths motors_rtos does not need. Configures only the main PLL.

Path: EXTAL 24 MHz -> MOSC -> PLL (x10 / 1) = 240 MHz -> SCKCR=0x21C21211. After this routine returns, ICLK is 120 MHz (half of production's 240 MHz) and PCLKA (the GPTW source) is 120 MHz, matching k_pclka_hz in libs/rx_hal/inc/rx72n_clock.h.

SPDX-License-Identifier: MIT

Definition in file clock.c.

4.1.2 Enumeration Type Documentation

4.1.2.1 clock_byte_constants_t

enum clock_byte_constants_t : uint8_t

Enumerator

k_sub_clock_stopped	SOSCCR: stop sub-clock oscillator
k_main_osc_enabled	MOSCCR: 0 = MOSC running
k_pll_enabled	PLLCR2: 0 = PLL running
k_oscovfsr_moovf	OSCOVFSR bit 0: MOSC stable
k_oscovfsr_plovf	OSCOVFSR bit 2: PLL stable

Definition at line 59 of file clock.c.

```
00059             : uint8_t {
00060   k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00061   k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
00062   k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
00063   k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00064   k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00065 } clock_byte_constants_t;
```

4.1.2.2 clock_extra_addrs_t

enum clock_extra_addrs_t : uintptr_t

Enumerator

k_packcr_addr	Peripheral clock source: 0=PLL

Definition at line 28 of file clock.c.

```
00028             : uintptr_t {
00029   k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00030 } clock_extra_addrs_t;
```

4.1.2.3 clock_sckcr_t

enum clock_sckcr_t : uint32_t

Enumerator

k_sckcr_value	

Definition at line 50 of file clock.c.

```
00050 : uint32_t {
00051 /*
00052  * SCKCR dividers (ICLK half-speed vs production):
00053  *   FCK=/4 (60), ICK=/2 (120), PSTOP0=1, PSTOP1=1, BCK=/4 (60),
00054  *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/2 (120), PCKD=/2 (120).
00055  */
00056 k_sckcr_value = 0x21C21211U,
00057 } clock_sckcr_t;
```

4.1.2.4 clock_timeout_t

enum clock_timeout_t : uint32_t

Enumerator

k_main_osc_poll_max	
k_pll_lock_poll_max	

Definition at line 67 of file clock.c.

```
00067 : uint32_t {
00068 /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00069  * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00070  * roughly 10 ms physical time regardless of CPU clock. The upper bound
00071  * just prevents an infinite spin if the crystal is missing. ~500k iters
00072  * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00073  * fixed 10-second busy-wait. */
00074 k_main_osc_poll_max  = 500000U,
00075 k_pll_lock_poll_max  = 500000U,
00076 } clock_timeout_t;
```

4.1.2.5 clock_word_constants_t

enum clock_word_constants_t : uint16_t

Enumerator

k_pllcr_mosc_x10	

Definition at line 35 of file clock.c.

```
00035 : uint16_t {
00036 /*
00037  * PLLCR layout:
00038  *   bits[1:0]  PLIDIV    : 00 = /1
00039  *   bit [4]    PLLSRCSEL : 0 = MOSC, 1 = HOCO
00040  *   bits[13:8] STC       : multiplier = (STC + 1) / 2
00041  *
00042  * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00043  * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00044  *
00045  * Production rx_clock_power_init.c uses k_pll_multiplier_10_div_1 = 0x1300.
00046  */
00047 k_pllcr_mosc_x10 = 0x1300U,
00048 } clock_word_constants_t;
```

4.1.3 Function Documentation

4.1.3.1 clock_init()

void clock_init (
    void)

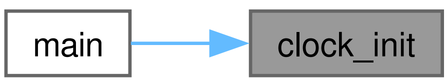
Definition at line 81 of file clock.c.

```
{
  volatile rx_system_regs_t *sys = system_regs();

  /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
  *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;

  /* Stop sub-clock oscillator -- not used on STAR. */
  sys->sosccr = k_sub_clock_stopped;

  /* Start MOSC (24 MHz external crystal). */
  sys->mosccr = k_main_osc_enabled;

  /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
   * settled (typically ~10 ms physical time) instead of waiting a fixed
   * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
  for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
    if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
      break;
    }
  }

  /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
  sys->pllcr  = k_pllcr_mosc_x10;
  sys->pllcr2 = k_pll_enabled;

  /* Wait for PLL lock (bounded). If we time out, the crystal is missing
   * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
   * on a dead clock. Stay on LOCO so the LEDs still respond and the user
   * can see we got past clock_init. */
  bool pll_locked = false;
  for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
    if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
      pll_locked = true;
      break;
    }
  }

  if (pll_locked) {
    /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
    *memwait_reg() = k_memwait_one_wait;

    /* Apply system clock dividers. */
    sys->sckcr = k_sckcr_value;

    /* Route UCLK from main PLL (UPLLSEL=0). */
    *((volatile uint16_t *)k_packcr_addr) = 0x0000U;

    /* Switch system clock source to PLL. */
    sys->sckcr3 = k_sckcr3_cksel_pll;
  }

  /* Re-lock PRCR. */
  *prcr_reg() = k_rx_prcr_lock;
}
```

References k_main_osc_enabled, k_main_osc_poll_max, k_oscovfsr_moovf, k_oscovfsr_plovf, k_packcr_addr, k_pll_enabled, k_pll_lock_poll_max, k_pllcr_mosc_x10, k_sckcr_value, and k_sub_clock_stopped.

Referenced by main().

Here is the caller graph for this function:



4.2 clock.c

Go to the documentation of this file.

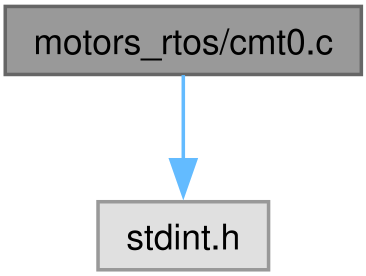
```c
/**
 * @file clock.c
 * @brief RX72N clock init for motors_rtos: MOSC 24 MHz crystal -> PLL 240 MHz
 *        -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
 *
 * @details
 * Self-contained reduced version of src/rx_clock_power_init.c, stripped of
 * the logging, asserts, simulator branches, and PPLL/USB clock paths
 * motors_rtos does not need. Configures only the main PLL.
 *
 * Path: EXTAL 24 MHz -> MOSC -> PLL (x10 / 1) = 240 MHz -> SCKCR=0x21C21211.
 * After this routine returns, ICLK is 120 MHz (half of production's 240 MHz)
 * and PCLKA (the GPTW source) is 120 MHz, matching k_pclka_hz in
 * libs/rx_hal/inc/rx72n_clock.h.
 *
 * SPDX-License-Identifier: MIT
 * @copyright Copyright (c) 2026 Locked Inc.
 */

#include <stdint.h>

#include "rx72n_regs.h"
#include "rx_register_protection.h"

/* ==========================================================================
 * Register addresses not exposed by rx_system_regs_t
 * ========================================================================== */
typedef enum : uintptr_t {
  k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
} clock_extra_addrs_t;

/* ==========================================================================
 * Bit / value constants (mirrors src/rx_clock_power_init.c)
 * ========================================================================== */
typedef enum : uint16_t {
  /*
   * PLLCR layout:
   *   bits[1:0]  PLIDIV     : 00 = /1
   *   bit [4]    PLLSRCSEL : 0 = MOSC, 1 = HOCO
   *   bits[13:8] STC        : multiplier = (STC + 1) / 2
   *
   * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
   * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
   *
   * Production rx_clock_power_init.c uses k_pll_multiplier_10_div_1 = 0x1300.
   */
  k_pllcr_mosc_x10 = 0x1300U,
} clock_word_constants_t;

typedef enum : uint32_t {
  /*
   * SCKCR dividers (ICLK half-speed vs production):
   *   FCK=/4 (60), ICK=/2 (120), PSTOP0=1, PSTOP1=1, BCK=/4 (60),
   *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/2 (120), PCKD=/2 (120).
   */
  k_sckcr_value = 0x21C21211U,
} clock_sckcr_t;

typedef enum : uint8_t {
  k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
  k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
  k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
  k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
  k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
} clock_byte_constants_t;

typedef enum : uint32_t {
  /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
   * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
   * roughly 10 ms physical time regardless of CPU clock. The upper bound
   * just prevents an infinite spin if the crystal is missing. ~500k iters
   * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
   * fixed 10-second busy-wait. */
  k_main_osc_poll_max  = 500000U,
  k_pll_lock_poll_max  = 500000U,
} clock_timeout_t;
```

```c

/* ===========================================================================
 * clock_init -- bring the chip from POR defaults to PLL 240 / PCKA 120 MHz.
 * ===========================================================================
 */
void clock_init(void)
{
  volatile rx_system_regs_t *sys = system_regs();

  /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
  *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;

  /* Stop sub-clock oscillator -- not used on STAR. */
  sys->sosccr = k_sub_clock_stopped;

  /* Start MOSC (24 MHz external crystal). */
  sys->mosccr = k_main_osc_enabled;

  /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
   * settled (typically ~10 ms physical time) instead of waiting a fixed
   * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
  for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
    if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
      break;
    }
  }

  /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
  sys->pllcr  = k_pllcr_mosc_x10;
  sys->pllcr2 = k_pll_enabled;

  /* Wait for PLL lock (bounded). If we time out, the crystal is missing
   * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
   * on a dead clock. Stay on LOCO so the LEDs still respond and the user
   * can see we got past clock_init. */
  bool pll_locked = false;
  for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
    if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
      pll_locked = true;
      break;
    }
  }

  if (pll_locked) {
    /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
    *memwait_reg() = k_memwait_one_wait;

    /* Apply system clock dividers. */
    sys->sckcr = k_sckcr_value;

    /* Route UCLK from main PLL (UPLLSEL=0). */
    *((volatile uint16_t *)k_packcr_addr) = 0x0000U;

    /* Switch system clock source to PLL. */
    sys->sckcr3 = k_sckcr3_cksel_pll;
  }

  /* Re-lock PRCR. */
  *prcr_reg() = k_rx_prcr_lock;
}
```

4.3 motors_rtos/cmt0.c File Reference

CMT0 ThreadX tick source for the PLL-clocked motors_rtos test.

#include <stdint.h>
Include dependency graph for cmt0.c:



Enumerations

- enum cmt0_hw_addr_t : uintptr_t {
k_cmstr0_addr = 0x00088000U , k_cmt0_cmcr = 0x00088002U , k_cmt0_cmcnt = 0x00088004U , k_cmt0_cmcor = 0x00088006U ,
k_icu_ir_base = 0x00087000U , k_icu_ier_base = 0x00087200U , k_icu_ipr_base = 0x00087300U , k_mstpcra_addr = 0x00080010U ,
k_prcr_addr = 0x000803FEU }
- enum cmt0_cfg_t : uint16_t { k_cmt0_cmcr_val = 0x0041U , k_cmt0_cmcor_val = 18749U , k_cmstr0_ch0_bit = 0x0001U }
- enum cmt0_icu_cfg_t : uint8_t {
k_vect_cmt0 = 28U , k_cmt0_priority = 3U , k_icu_ier_cmt0 = 3U , k_icu_ier_cmt0_bit = 4U ,
k_mstpcra_cmt0_bit = 15U }
- enum cmt0_prcr_t : uint16_t { k_prcr_unlock_prc1 = 0xA502U , k_prcr_lock = 0xA500U }

Functions

- static volatile uint16_t ∗ cmstr0 (void)
- static volatile uint16_t ∗ cmt0_cmcr (void)
- static volatile uint16_t ∗ cmt0_cmcnt (void)
- static volatile uint16_t ∗ cmt0_cmcor (void)
- static volatile uint8_t ∗ icu_ir (uint8_t vect)
- static volatile uint8_t ∗ icu_ier (uint8_t idx)
- static volatile uint8_t ∗ icu_ipr (uint8_t vect)
- static volatile uint32_t ∗ mstpcra (void)
- static volatile uint16_t ∗ prcr (void)
- void cmt0_init (void)

Variables

- volatile uint32_t g_cmt0_isr_count = 0U

4.3.1 Detailed Description

CMT0 ThreadX tick source for the PLL-clocked motors_rtos test.

Drives ThreadX's 100 Hz system tick via CMT0 CMI0 (vector 28). This variant targets the PLL clock tree (PCLKB = 60 MHz), not the LOCO default used by blinky_rtos – motors_rtos calls clock_init() before tx_kernel_enter() so CMT0 sees the full 60 MHz PCLKB.

Math: PCLKB / 32 = 60 MHz / 32 = 1.875 MHz; CMCOR = 18749 -> 100 Hz.

The ISR must:

1. Clear the ICU IR flag (otherwise the IRQ re-fires immediately).
2. Call _tx_timer_interrupt() so ThreadX advances its clock.

vectors.S wires slot 28 to _cmt0_isr – GCC prefixes this file's cmt0_isr symbol with an underscore on link, matching that reference.

SPDX-License-Identifier: MIT

Copyright

Definition in file cmt0.c.

4.3.2 Enumeration Type Documentation

4.3.2.1 cmt0_cfg_t

enum cmt0_cfg_t : uint16_t

Enumerator

k_cmt0_cmcr_val	CKS=01 (PCLKB/32), CMIE=1
k_cmt0_cmcor_val	60 MHz / 32 / 100 Hz - 1
k_cmstr0_ch0_bit	

Definition at line 43 of file cmt0.c.

```
00043             : uint16_t {
00044     /*
00045      * CMCR layout:
00046      *   bits[1:0] CKS : 00=PCLK/8, 01=PCLK/32, 10=PCLK/128, 11=PCLK/512
00047      *   bit [6]   CMIE: 0=disabled, 1=enabled
00048      */
00049     k_cmt0_cmcr_val  = 0x0041U, /**< CKS=01 (PCLKB/32), CMIE=1 */
00050     k_cmt0_cmcor_val = 18749U,  /**< 60 MHz / 32 / 100 Hz - 1 */
00051     k_cmstr0_ch0_bit = 0x0001U,
00052 } cmt0_cfg_t;
```

4.3.2.2 cmt0_hw_addr_t

enum cmt0_hw_addr_t : uintptr_t

Enumerator

k_cmstr0_addr	CMT start register (CH0/CH1 share)
k_cmt0_cmcr	CMT0 control register
k_cmt0_cmcnt	CMT0 counter
k_cmt0_cmcor	CMT0 compare match register
k_icu_ir_base	
k_icu_ier_base	

k_icu_ipr_base	
k_mstpcra_addr	MSTPCRA (bit 15 = CMT0/CMT1 unit)
k_prcr_addr	PRCR protection register

Definition at line 29 of file cmt0.c.

```
00029             : uintptr_t {
00030     k_cmstr0_addr = 0x00088000U, /**< CMT start register (CH0/CH1 share) */
00031     k_cmt0_cmcr   = 0x00088002U, /**< CMT0 control register */
00032     k_cmt0_cmcnt  = 0x00088004U, /**< CMT0 counter */
00033     k_cmt0_cmcor  = 0x00088006U, /**< CMT0 compare match register */
00034
00035     k_icu_ir_base  = 0x00087000U,
00036     k_icu_ier_base = 0x00087200U,
00037     k_icu_ipr_base = 0x00087300U,
00038
00039     k_mstpcra_addr = 0x00080010U, /**< MSTPCRA (bit 15 = CMT0/CMT1 unit) */
00040     k_prcr_addr    = 0x000803FEU, /**< PRCR protection register */
00041 } cmt0_hw_addr_t;
```

4.3.2.3 cmt0_icu_cfg_t

enum cmt0_icu_cfg_t : uint8_t

Enumerator

k_vect_cmt0	
k_cmt0_priority	
k_icu_ier_cmt0	IER[28 / 8] = IER[3]
k_icu_ier_cmt0_bit	28 % 8 = 4
k_mstpcra_cmt0_bit	

Definition at line 54 of file cmt0.c.

```
00054             : uint8_t {
00055     k_vect_cmt0        = 28U,
00056     k_cmt0_priority    = 3U,
00057     k_icu_ier_cmt0     = 3U, /**< IER[28 / 8] = IER[3] */
00058     k_icu_ier_cmt0_bit = 4U, /**< 28 % 8 = 4 */
00059     k_mstpcra_cmt0_bit = 15U,
00060 } cmt0_icu_cfg_t;
```

4.3.2.4 cmt0_prcr_t

enum cmt0_prcr_t : uint16_t

Enumerator

k_prcr_unlock_prc1	PRKEY=A5, PRC1=1 (CGC access)
k_prcr_lock	

Definition at line 62 of file cmt0.c.

```
00062             : uint16_t {
00063     k_prcr_unlock_prc1 = 0xA502U, /**< PRKEY=A5, PRC1=1 (CGC access) */
00064     k_prcr_lock        = 0xA500U,
00065 } cmt0_prcr_t;
```

4.3.3 Function Documentation

4.3.3.1 cmstr0()

volatile uint16_t * cmstr0 (
void) [inline], [static]

Definition at line 70 of file cmt0.c.

```
00070 { return (volatile uint16_t *)k_cmstr0_addr; }
```

References k_cmstr0_addr.

Referenced by cmt0_init().

Here is the caller graph for this function:

4.3.3.2 cmt0_cmcnt()

volatile uint16_t * cmt0_cmcnt (
void) [inline], [static]

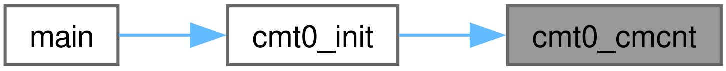
Definition at line 72 of file cmt0.c.

```
00072 { return (volatile uint16_t *)k_cmt0_cmcnt; }
```

References k_cmt0_cmcnt.

Referenced by cmt0_init().

Here is the caller graph for this function:



4.3.3.3 cmt0_cmcor()

volatile uint16_t * cmt0_cmcor (
void) [inline], [static]

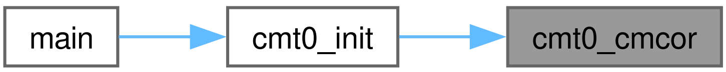
Definition at line 73 of file cmt0.c.

```
00073 { return (volatile uint16_t *)k_cmt0_cmcor; }
```

References k_cmt0_cmcor.

Referenced by cmt0_init().

Here is the caller graph for this function:



4.3.3.4 cmt0_cmcr()

volatile uint16_t ∗ cmt0_cmcr (
 void) [inline], [static]

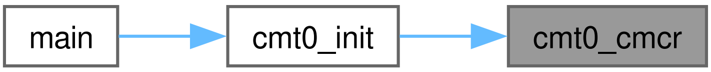
Definition at line 71 of file cmt0.c.

```
00071 { return (volatile uint16_t *)k_cmt0_cmcr; }
```

References k_cmt0_cmcr.

Referenced by cmt0_init().

Here is the caller graph for this function:



4.3.3.5 cmt0_init()

void cmt0_init (
 void)

Definition at line 112 of file cmt0.c.

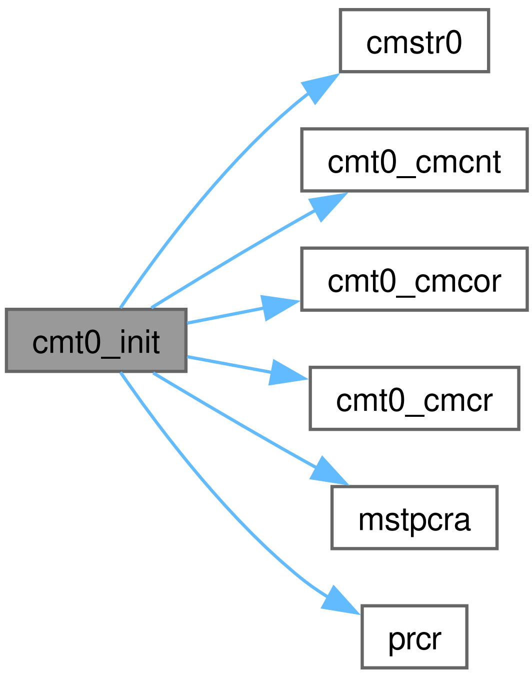
```
00113 {
00114     /* Release CMT0 from module stop (MSTPCRA bit 15 controls the CMT0/CMT1 unit). */
00115     *prcr() = k_prcr_unlock_prc1;
00116     *mstpcra() &= ~(uint32_t)(1UL « (uint32_t)k_mstpcra_cmt0_bit);
00117     *prcr() = k_prcr_lock;
00118
00119     /* Stop CMT0, configure, then start. */
00120     *cmstr0()    &= (uint16_t)~(uint16_t)k_cmstr0_ch0_bit;
00121     *cmt0_cmcr()  = (uint16_t)k_cmt0_cmcr_val;
00122     *cmt0_cmcor() = (uint16_t)k_cmt0_cmcor_val;
00123     *cmt0_cmcnt() = 0U;
00124
00125     /* Direct-address writes -- bypass any macro/accessor doubt.  Addresses
00126      * verified against libs/rx_hal/inc/rx72n_icu_regs.h: IR/IER/IPR are
00127      * uint8_t arrays at offsets 0x000/0x200/0x300 from ICU base 0x00087000.
00128      *   IR[28]  = 0x0008_701C
00129      *   IER[3]  = 0x0008_7203, bit 4 = vector 28
00130      *   IPR[28] = 0x0008_731C
00131      * Priority raised from 3 to 10 to match production (rx72n_icu_regs.h
00132      * example doc). */
00133     /* ICU setup for CMT0_CMI0 (vector 28).  Direct-address writes document
00134      * each register explicitly; indices verified against
00135      * libs/rx_hal/inc/rx72n_icu_regs.h (@0x000 IR, @0x200 IER, @0x300 IPR).
00136      *
00137      * IMPORTANT -- RX72N IPR INDEX IS *NOT* THE VECTOR NUMBER.
00138      * The HAL's example code `icu_regs->ipr[28] = 10‘ at
00139      * rx72n_icu_regs.h:318 is WRONG for CMT0_CMI0.  IPR[] on RX72N is
00140      * sparse-mapped per a hardware lookup table in the RX72N HW manual
00141      * section 14 (ICU).  Vector 28 (CMT0_CMI0) maps to **IPR[4]**, not
00142      * IPR[28].  Writes to IPR[28] (0x8731C) are silently dropped -- reads
00143      * return 0 regardless of what was written.  Confirmed empirically:
00144      * writing 10 to IPR[4] made the ISR fire (g_cmt0_isr_count advanced
00145      * from 0 to 604+ in ~6 sec); writing the same to IPR[28] had zero
00146      * effect.  See also eclipse-threadx/rtos-docs and Renesas community
00147      * threads referenced in the matching RCA session notes.
00148      *
00149      * We intentionally keep BOTH writes for now: IPR[4] is the real
00150      * priority register; IPR[28] is a no-op we leave in so future readers
00151      * can trivially see the wrong-address trap that cost us a session,
00152      * with this comment serving as the "why".  Remove the IPR[28] write
00153      * after this test is known-stable on hardware. */
00154     *(volatile uint8_t *)0x0008701CU = 0U;   /* IR[28]  -- clear pending */
00155     *(volatile uint8_t *)0x00087304U = 10U;  /* IPR[4]  -- real priority register for vector 28 on RX72N */
00156     *(volatile uint8_t *)0x0008731CU = 10U;  /* IPR[28] -- NO-OP, intentionally kept as a tombstone (see comment above) */
00157     *(volatile uint8_t *)0x00087203U |= (uint8_t)(1U « 4); /* IER[3] bit 4 enable */
```

```

    *cmstr0() |= (uint16_t)k_cmstr0_ch0_bit;

    /* Globally enable interrupts -- ThreadX expects PSW.I = 1 before kernel entry. */
    __asm__ volatile("setpsw i");
}
```

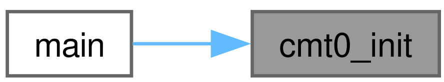
References cmstr0(), cmt0_cmcnt(), cmt0_cmcor(), cmt0_cmcr(), k_cmstr0_ch0_bit, k_cmt0_cmcor_val, k_cmt0_cmcr_val, k_mstpcra_cmt0_bit, k_prcr_lock, k_prcr_unlock_prc1, mstpcra(), and prcr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.6 icu_ier()

volatile uint8_t ∗ icu_ier (
    uint8_t idx) [inline], [static]

Definition at line 79 of file cmt0.c.

```
{
    return (volatile uint8_t *)(k_icu_ier_base + idx);
}
```

References k_icu_ier_base.

4.3.3.7 icu_ipr()

volatile uint8_t * icu_ipr (
uint8_t vect) [inline], [static]

Definition at line 83 of file cmt0.c.

```
00084 {
00085     return (volatile uint8_t *)(k_icu_ipr_base + vect);
00086 }
```

References k_icu_ipr_base.

4.3.3.8 icu_ir()

volatile uint8_t * icu_ir (
uint8_t vect) [inline], [static]

Definition at line 75 of file cmt0.c.

```
00076 {
00077     return (volatile uint8_t *)(k_icu_ir_base + vect);
00078 }
```

References k_icu_ir_base.

4.3.3.9 mstpcra()

volatile uint32_t * mstpcra (
void) [inline], [static]

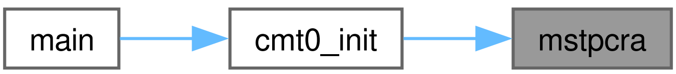
Definition at line 88 of file cmt0.c.

```
00089 {
00090     return (volatile uint32_t *)k_mstpcra_addr;
00091 }
```

References k_mstpcra_addr.

Referenced by cmt0_init().

Here is the caller graph for this function:



4.3.3.10 prcr()

volatile uint16_t * prcr (
void) [inline], [static]

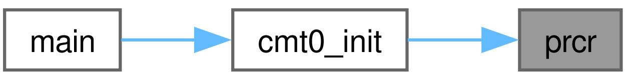
Definition at line 92 of file cmt0.c.

```
00093 {
00094     return (volatile uint16_t *)k_prcr_addr;
00095 }
```

References k_prcr_addr.

Referenced by cmt0_init().

Here is the caller graph for this function:



4.3.4 Variable Documentation

4.3.4.1 g_cmt0_isr_count

volatile uint32_t g_cmt0_isr_count = 0U

from cmt0.c – verifies CMT0 tick ISR is firing

Definition at line 107 of file cmt0.c.

Referenced by encoder_task_entry().

4.4 cmt0.c

Go to the documentation of this file.

```
00001 /**
00002  * @file cmt0.c
00003  * @brief CMT0 ThreadX tick source for the PLL-clocked motors_rtos test.
00004  *
00005  * @details
00006  * Drives ThreadX's 100 Hz system tick via CMT0 CMI0 (vector 28).  This
00007  * variant targets the PLL clock tree (PCLKB = 60 MHz), not the LOCO
00008  * default used by blinky_rtos -- motors_rtos calls clock_init() before
00009  * tx_kernel_enter() so CMT0 sees the full 60 MHz PCLKB.
00010  *
00011  * Math: PCLKB / 32 = 60 MHz / 32 = 1.875 MHz; CMCOR = 18749 -> 100 Hz.
00012  *
00013  * The ISR must:
00014  *   1. Clear the ICU IR flag (otherwise the IRQ re-fires immediately).
00015  *   2. Call _tx_timer_interrupt() so ThreadX advances its clock.
00016  *
00017  * vectors.S wires slot 28 to `_cmt0_isr` -- GCC prefixes this file's
00018  * cmt0_isr symbol with an underscore on link, matching that reference.
00019  *
00020  * SPDX-License-Identifier: MIT
00021  * @copyright Copyright (c) 2026 Locked Inc.
00022  */
00023
00024 #include <stdint.h>
00025
00026 /*
        ==========================================================================
00027  * Register addresses (RX72N HW manual R01UH0824EJ0111, Ch 15 CMT + Ch 13 ICU)
00028  *
        ==========================================================================
        */
00029 typedef enum : uintptr_t {
00030     k_cmstr0_addr = 0x00088000U, /**< CMT start register (CH0/CH1 share) */
00031     k_cmt0_cmcr   = 0x00088002U, /**< CMT0 control register */
00032     k_cmt0_cmcnt  = 0x00088004U, /**< CMT0 counter */
00033     k_cmt0_cmcor  = 0x00088006U, /**< CMT0 compare match register */
00034
00035     k_icu_ir_base  = 0x00087000U,
00036     k_icu_ier_base = 0x00087200U,
00037     k_icu_ipr_base = 0x00087300U,
00038
00039     k_mstpcra_addr = 0x00080010U, /**< MSTPCRA (bit 15 = CMT0/CMT1 unit) */
00040     k_prcr_addr    = 0x000803FEU, /**< PRCR protection register */
00041 } cmt0_hw_addr_t;
00042
00043 typedef enum : uint16_t {
00044     /*
00045      * CMCR layout:
00046      *   bits[1:0] CKS : 00=PCLK/8, 01=PCLK/32, 10=PCLK/128, 11=PCLK/512
00047      *   bit [6]   CMIE: 0=disabled, 1=enabled
00048      */
00049     k_cmt0_cmcr_val  = 0x0041U, /**< CKS=01 (PCLKB/32), CMIE=1 */
00050     k_cmt0_cmcor_val = 18749U,  /**< 60 MHz / 32 / 100 Hz - 1 */
00051     k_cmstr0_ch0_bit = 0x0001U,
00052 } cmt0_cfg_t;
00053
00054 typedef enum : uint8_t {
00055     k_vect_cmt0         = 28U,
00056     k_cmt0_priority     = 3U,
00057     k_icu_ier_cmt0      = 3U, /**< IER[28 / 8] = IER[3] */
```

```
    k_icu_ier_cmt0_bit  = 4U, /**< 28 % 8 = 4 */
    k_mstpcra_cmt0_bit  = 15U,
} cmt0_icu_cfg_t;

typedef enum : uint16_t {
    k_prcr_unlock_prc1 = 0xA502U, /**< PRKEY=A5, PRC1=1 (CGC access) */
    k_prcr_lock        = 0xA500U,
} cmt0_prcr_t;

/*
   ============================================================================
 * Register accessors
 *
   ============================================================================
   */
static inline volatile uint16_t *cmstr0(void)    { return (volatile uint16_t *)k_cmstr0_addr; }
static inline volatile uint16_t *cmt0_cmcr(void) { return (volatile uint16_t *)k_cmt0_cmcr; }
static inline volatile uint16_t *cmt0_cmcnt(void){ return (volatile uint16_t *)k_cmt0_cmcnt; }
static inline volatile uint16_t *cmt0_cmcor(void){ return (volatile uint16_t *)k_cmt0_cmcor; }

static inline volatile uint8_t *icu_ir(uint8_t vect)
{
    return (volatile uint8_t *)(k_icu_ir_base + vect);
}
static inline volatile uint8_t *icu_ier(uint8_t idx)
{
    return (volatile uint8_t *)(k_icu_ier_base + idx);
}
static inline volatile uint8_t *icu_ipr(uint8_t vect)
{
    return (volatile uint8_t *)(k_icu_ipr_base + vect);
}

static inline volatile uint32_t *mstpcra(void)
{
    return (volatile uint32_t *)k_mstpcra_addr;
}
static inline volatile uint16_t *prcr(void)
{
    return (volatile uint16_t *)k_prcr_addr;
}


/*
   ============================================================================
 * CMT0 ISR (wired at vectors.S slot 28).  GCC's __attribute__((interrupt))
 * generates the RTE-terminated prologue/epilogue the RX expects.
 *
   ============================================================================
   */
/* Exposed to main.c for the diagnostic CSV column -- counts how many times
 * the CMT0 compare-match ISR has fired.  Incremented by the assembly ISR
 * in vectors.S (which wraps _tx_timer_interrupt in context_save/restore so
 * that timer-based task wakeups actually cause a context switch).  See the
 * `__cmt0_isr:‘ block in vectors.S for the full ISR implementation. */
volatile uint32_t g_cmt0_isr_count = 0U;

/*
   ============================================================================
 * cmt0_init -- call once, before tx_kernel_enter().
 *
   ============================================================================
   */
void cmt0_init(void)
{
    /* Release CMT0 from module stop (MSTPCRA bit 15 controls the CMT0/CMT1 unit). */
    *prcr() = k_prcr_unlock_prc1;
    *mstpcra() &= ~(uint32_t)(1UL « (uint32_t)k_mstpcra_cmt0_bit);
    *prcr() = k_prcr_lock;

    /* Stop CMT0, configure, then start. */
    *cmstr0()    &= (uint16_t)~(uint16_t)k_cmstr0_ch0_bit;
    *cmt0_cmcr()  = (uint16_t)k_cmt0_cmcr_val;
    *cmt0_cmcor() = (uint16_t)k_cmt0_cmcor_val;
    *cmt0_cmcnt() = 0U;

    /* Direct-address writes -- bypass any macro/accessor doubt.  Addresses
     * verified against libs/rx_hal/inc/rx72n_icu_regs.h: IR/IER/IPR are
     * uint8_t arrays at offsets 0x000/0x200/0x300 from ICU base 0x00087000.
     *   IR[28]  = 0x0008_701C
     *   IER[3]  = 0x0008_7203, bit 4 = vector 28
     *   IPR[28] = 0x0008_731C
     * Priority raised from 3 to 10 to match production (rx72n_icu_regs.h
     * example doc). */
    /* ICU setup for CMT0_CMI0 (vector 28).  Direct-address writes document
     * each register explicitly; indices verified against
     * libs/rx_hal/inc/rx72n_icu_regs.h (@0x000 IR, @0x200 IER, @0x300 IPR).
```

```
     *
     * IMPORTANT -- RX72N IPR INDEX IS *NOT* THE VECTOR NUMBER.
     * The HAL's example code `icu_regs->ipr[28] = 10‘ at
     * rx72n_icu_regs.h:318 is WRONG for CMT0_CMI0.  IPR[] on RX72N is
     * sparse-mapped per a hardware lookup table in the RX72N HW manual
     * section 14 (ICU).  Vector 28 (CMT0_CMI0) maps to **IPR[4]**, not
     * IPR[28].  Writes to IPR[28] (0x8731C) are silently dropped -- reads
     * return 0 regardless of what was written.  Confirmed empirically:
     * writing 10 to IPR[4] made the ISR fire (g_cmt0_isr_count advanced
     * from 0 to 604+ in ~6 sec); writing the same to IPR[28] had zero
     * effect.  See also eclipse-threadx/rtos-docs and Renesas community
     * threads referenced in the matching RCA session notes.
     *
     * We intentionally keep BOTH writes for now: IPR[4] is the real
     * priority register; IPR[28] is a no-op we leave in so future readers
     * can trivially see the wrong-address trap that cost us a session,
     * with this comment serving as the "why".  Remove the IPR[28] write
     * after this test is known-stable on hardware. */
    *(volatile uint8_t *)0x0008701CU = 0U;   /* IR[28]  -- clear pending */
    *(volatile uint8_t *)0x00087304U = 10U;  /* IPR[4]  -- real priority register for vector 28 on RX72N */
    *(volatile uint8_t *)0x0008731CU = 10U;  /* IPR[28] -- NO-OP, intentionally kept as a tombstone (see comment above) */
    *(volatile uint8_t *)0x00087203U |= (uint8_t)(1U « 4); /* IER[3] bit 4 enable */

    *cmstr0() |= (uint16_t)k_cmstr0_ch0_bit;

    /* Globally enable interrupts -- ThreadX expects PSW.I = 1 before kernel entry. */
    __asm__ volatile("setpsw i");
}
```

4.5 motors_rtos/linker.ld File Reference

4.6 linker.ld

Go to the documentation of this file.

```
/*
 * blinky_rtos/linker.ld -- Linker script for ThreadX-based multi-LED breathe.
 *
 * RX72N memory map (R5F572NNHxFB):
 *   Internal RAM  : 0x00000004 - 0x0007FFFF  (512 KB, first 4 bytes reserved)
 *   Internal ROM  : 0xFFE00000 - 0xFFFFFFFF  (2 MB)
 *
 * Fixed hardware vector locations:
 *   0xFFFFFF80 - 0xFFFFFFFC  Exception vector table (EXTB)
 *   0xFFFFFFFC - 0xFFFFFFFF  Reset vector (points to startup entry)
 *
 * INTB is loaded by startup.S to point at _rvectors_start (in ROM).
 *
 * Renesas-syntax sections from ThreadX RXv3 port (tx_timer_interrupt.S etc.):
 *   P, P_1  -- code sections (.SECTION P,CODE)
 *   B, B_1  -- BSS sections  (.SECTION B,DATA)
 *   D, D_1  -- initialised data sections
 */

MEMORY
{
    RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC   /* 512 KB - 4 bytes */
    ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000  /* 2 MB             */
}

SECTIONS
{
    /* Code -- explicit ROM base so section VMA is anchored correctly.
     * *(P) captures Renesas-assembler code sections from ThreadX port files. */
    .text 0xFFE00000 : AT(0xFFE00000)
    {
        *(.text*)
        *(P)
        *(P_1)
        *(.rodata*)
        *(C)
        *(C_1)
        *(C_2)
        etext = .;
    } > ROM

    /*
     * Relocatable vector table -- startup.S loads INTB with this address.
```

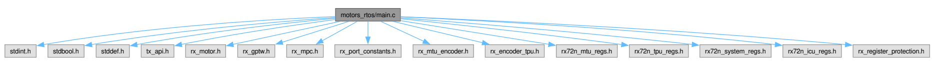
```
 * Must be 4-byte aligned; 32 entries = 128 bytes.
 * No AT() needed: follows .text in ROM, VMA and LMA are the same.
 */
.rvectors ALIGN(4) :
{
    __rvectors_start = .;
    KEEP(*(.rvectors))
    __rvectors_end = .;
} > ROM

/*
 * Initialised data -- LMA stays in ROM (image stored here), VMA in RAM.
 * startup.S copies from _mdata (ROM) to _data.._edata (RAM) on boot.
 * Required for ThreadX: ___tx_thread_system_state starts as 0xF0F0F0F0.
 */
_mdata = .;
.data : AT(_mdata)
{
    _data = .;
    *(.data*)
    *(D)
    *(D_1)
    *(D_2)
    _edata = .;
} > RAM

/* BSS -- ThreadX needs this zeroed; startup.S handles it.
 * B/B_1/B_2 are Renesas-assembler BSS sections from ThreadX port files. */
.bss :
{
    _bss = .;
    *(.bss*)
    *(COMMON)
    *(B)
    *(B_1)
    *(B_2)
    _ebss = .;
    . = ALIGN(4);
    _end = .;           /* ThreadX tx_initialize_low_level reads _end */
} > RAM AT>RAM

/*
 * Stacks -- USP grows down from top of RAM, ISP 2 KB below that.
 * ThreadX uses the interrupt stack pointer (ISP) for ISR execution.
 */
_ustack = ORIGIN(RAM) + LENGTH(RAM);
_istack = _ustack - 0x800;

/*
 * Exception vector table (EXTB) at fixed hardware address 0xFFFFFF80.
 */
.exvectors 0xFFFFFF80 : AT(0xFFFFFF80)
{
    LONG(0xFFFFFFFF);   /* BUSERR */
    LONG(0xFFFFFFFF);   /* reserved */
    LONG(0xFFFFFFFF);   /* FIFERR */
    LONG(0xFFFFFFFF);   /* FRDYI */
    LONG(0xFFFFFFFF);   /* reserved */
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
    LONG(0xFFFFFFFF);
} > ROM

/* Fixed reset vector at 0xFFFFFFFC */
```

```
.fvectors 0xFFFFFFFC : AT(0xFFFFFFFC)
{
    KEEP(*(.fvectors))
} > ROM
}
```

4.7 motors_rtos/main.c File Reference

motors_rtos – motors + encoders under Azure RTOS ThreadX.

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "tx_api.h"
#include "rx_motor.h"
#include "rx_gptw.h"
#include "rx_mpc.h"
#include "rx_port_constants.h"
#include "rx_mtu_encoder.h"
#include "rx_encoder_tpu.h"
#include "rx72n_mtu_regs.h"
#include "rx72n_tpu_regs.h"
#include "rx72n_system_regs.h"
#include "rx72n_icu_regs.h"
#include "rx_register_protection.h"
Include dependency graph for main.c:



Data Structures

- struct motor_wiring_t

Macros

- #define REG8(a)
- #define REG8_AT(a)
- #define REG16_AT(a)

Enumerations

- enum port_reg_base_t : uintptr_t { k_port_pdr_base = 0x0008C000U , k_port_podr_base = 0x0008C020U , k_port_pmr_base = 0x0008C060U }
- enum status_led_pins_t : uint8_t {
  k_port_7 = 7 , k_port_a = 10 , k_port_b = 11 , k_port_e = 14 ,
  k_port_6 = 6 , k_bit_led_heartbeat = 7 , k_bit_led_init_done = 0 , k_bit_led_motor_run = 1 ,
  k_bit_led_enc_tick = 2 , k_bit_led_motor_alive = 1 }
- enum motor_const_t : uint8_t { k_motor_count = 4 }
- enum delay_const_t : uint32_t { k_iters_per_ms = 40000U }
- enum pwm_const_t : uint32_t { k_pwm_freq_hz = 20000U , k_deadtime_ns = 1000U }
- enum sweep_const_t : uint8_t { k_sweep_step_count = 21U }
- enum rtos_stack_t : uint16_t { k_motor_task_stack_sz = 2048U , k_encoder_task_stack_sz = 2048U }
- enum rtos_prio_t : uint8_t { k_encoder_task_priority = 9U , k_motor_task_priority = 10U , k_motor_sleep_ticks = 50U , k_encoder_sleep_ticks = 10U }

Functions

- void clock_init (void)
- void cmt0_init (void)
- void sci9_debug_init (void)
- void sci9_debug_putc (char c)
- void sci9_debug_puts (const char *s)
- void sci9_debug_puthex16 (uint16_t v)
- void sci9_debug_puthex32 (uint32_t v)
- static volatile uint8_t * port_pdr (uint8_t port)
- static volatile uint8_t * port_podr (uint8_t port)
- static volatile uint8_t * port_pmr (uint8_t port)
- static void gpio_set_output (uint8_t port, uint8_t pin)
- static void gpio_write (uint8_t port, uint8_t pin, bool high)
- static void gpio_toggle (uint8_t port, uint8_t pin)
- static void leds_init (void)
- static void busy_wait_ms (uint32_t ms)
- static void motor_drv_gpio_init (void)
- static bool motor_pwm_init (rx_motor_handle_t handles[k_motor_count])
- static bool encoders_init (void)
- static uint16_t encoder_read_raw (uint8_t motor_idx)
- static void print_s16_dec (int16_t v)
- static void motor_task_entry (ULONG arg)
- static void encoder_task_entry (ULONG arg)
- void tx_application_define (void *first_unused_memory)
- static void error_park (const char *msg)
- int main (void)

Variables

- static const motor_wiring_t k_motors [k_motor_count]
- static const int8_t k_sweep_table [k_sweep_step_count]
- static TX_THREAD s_motor_tcb
- static TX_THREAD s_encoder_tcb
- static uint8_t s_motor_stack [k_motor_task_stack_sz]
- static uint8_t s_encoder_stack [k_encoder_task_stack_sz]
- static rx_motor_handle_t s_motors [k_motor_count]
- static volatile int8_t s_current_duty = 0
- static volatile uint32_t g_motor_task_entries = 0U
- static volatile uint32_t g_motor_task_loops = 0U
- static volatile uint32_t g_motor_task_loops_after = 0U
- static volatile uint32_t g_motor_create_status = 0xFFFFFFFFU
- static volatile uint32_t g_encoder_create_status = 0xFFFFFFFFU
- volatile uint32_t g_cmt0_isr_count
- UINT _tx_thread_preempt_disable
- ULONG _tx_thread_system_state
- TX_THREAD * _tx_thread_current_ptr

4.7.1 Detailed Description

motors_rtos – motors + encoders under Azure RTOS ThreadX.

Two-task ThreadX bring-up of libs/rx_motor + libs/rx_encoder, proving they coexist with the scheduler. Mirrors encoder_test's open-loop spin + count logic but splits the work across two cooperating tasks:

motor_task (priority 10) – every 500 ms, advances a duty-sweep state machine. All 4 motors take the same duty, with per-motor direction_sign applied so FL/BL spin the same wheel direction as FR/BR despite the reversed wiring.

encoder_task (priority 9) – every 100 ms, reads all 4 encoder TCNTs, prints a one-line CSV over SCI9 (115200, /dev/ttyACM0 on the host), and toggles the PA7/D9 heartbeat LED. Higher priority than motor_task so the console stays responsive even when motor_task is awake.

Boot sequence (main):

1. clock_init() – PLL 240 MHz / PCLKA 120 / PCLKB 60
2. sci9_debug_init() – console up; banner printed immediately
3. motor_drv_gpio_init – DRVOFF high, nSLEEP high, tWAKE 10 ms, DRVOFF low
4. motor_pwm_init – rx_motor_init x4 at 20 kHz, 1 us deadtime
5. encoders_init – MTU1/MTU2/TPU1/TPU2 phase counting mode 1
6. cmt0_init – 100 Hz ThreadX tick + setpsw i
7. tx_kernel_enter – hands off forever

Per memory project_motor_pwm_hal_bugs.md: the production rx_mpc / rx_gptw stack has latent issues writing PFS for MTCLK/TCLK encoder pins, so the encoders_init() routine here mirrors encoder_test's direct-register workaround rather than calling rx_mpc_set_mtu_encoder() / rx_mpc_set_tpu_encoder().

SPDX-License-Identifier: MIT

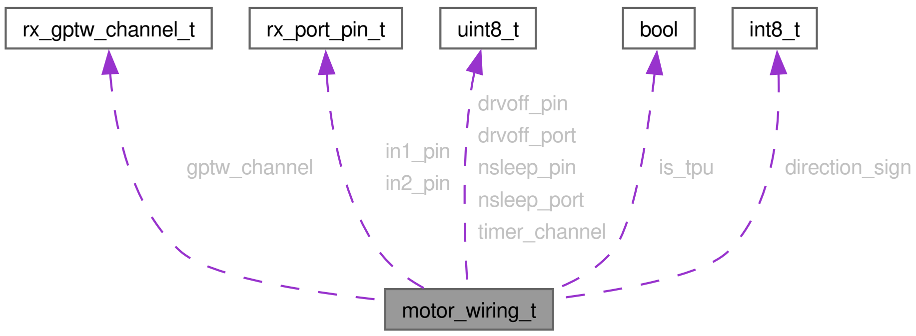
Definition in file main.c.

4.7.2 Data Structure Documentation

4.7.2.1 struct motor_wiring_t

Definition at line 159 of file main.c.

Collaboration diagram for motor_wiring_t:



Data Fields

int8_t	direction_sign	
uint8_t	drvoff_pin	
uint8_t	drvoff_port	
rx_gptw_channel_t	gptw_channel	
rx_port_pin_t	in1_pin	
rx_port_pin_t	in2_pin	
bool	is_tpu	
uint8_t	nsleep_pin	
uint8_t	nsleep_port	
uint8_t	timer_channel	

4.7.3 Macro Definition Documentation

4.7.3.1 REG16_AT

```
#define REG16_AT(
                a)
```

Value:
```
(*(volatile uint16_t *)(uintptr_t)(a))
```

Definition at line 268 of file main.c.

Referenced by encoder_read_raw().

4.7.3.2 REG8

```
#define REG8(
                a)
```

Value:
```
(*(volatile uint8_t *)(uintptr_t)(a))
```

Definition at line 71 of file main.c.

Referenced by port_pdr(), port_pmr(), and port_podr().

4.7.3.3 REG8_AT

```
#define REG8_AT(
                    a)
```

Value:

```
(*(volatile uint8_t *)(uintptr_t)(a))
```

Definition at line 267 of file main.c.

Referenced by encoders_init().

4.7.4 Enumeration Type Documentation

4.7.4.1 delay_const_t

enum delay_const_t : uint32_t

Enumerator

k_iters_per_ms	

Definition at line 200 of file main.c.

```
00200              : uint32_t {
00201     k_iters_per_ms = 40000U,
00202 } delay_const_t;
```

4.7.4.2 motor_const_t

enum motor_const_t : uint8_t

Enumerator

k_motor_count	

Definition at line 172 of file main.c.

```
00172              : uint8_t {
00173     k_motor_count = 4,
00174 } motor_const_t;
```

4.7.4.3 port_reg_base_t

enum port_reg_base_t : uintptr_t

Enumerator

k_port_pdr_base	
k_port_podr_base	

k_port_pmr_base	

Definition at line 73 of file main.c.

```
00073              : uintptr_t {
00074     k_port_pdr_base  = 0x0008C000U,
00075     k_port_podr_base = 0x0008C020U,
00076     k_port_pmr_base  = 0x0008C060U,
00077 } port_reg_base_t;
```

4.7.4.4 pwm_const_t

enum pwm_const_t : uint32_t

Enumerator

k_pwm_freq_hz	
k_deadtime_ns	

Definition at line 235 of file main.c.

```
00235              : uint32_t {
00236     k_pwm_freq_hz  = 20000U,
00237     k_deadtime_ns  = 1000U,
00238 } pwm_const_t;
```

4.7.4.5 rtos_prio_t

enum rtos_prio_t : uint8_t

Enumerator

k_encoder_task_priority	higher priority: prints CSV, keeps console responsive
k_motor_task_priority	lower priority: sweeps duty, preempted by encoder
k_motor_sleep_ticks	50 * 10 ms = 500 ms
k_encoder_sleep_ticks	10 * 10 ms = 100 ms

Definition at line 406 of file main.c.

```
00406              : uint8_t {
00407     k_encoder_task_priority = 9U,  /**< higher priority: prints CSV, keeps console responsive */
00408     k_motor_task_priority   = 10U, /**< lower priority: sweeps duty, preempted by encoder */
00409     k_motor_sleep_ticks     = 50U, /**< 50 * 10 ms = 500 ms */
00410     k_encoder_sleep_ticks   = 10U, /**< 10 * 10 ms = 100 ms */
00411 } rtos_prio_t;
```

4.7.4.6 rtos_stack_t

enum rtos_stack_t : uint16_t

Enumerator

k_motor_task_stack_sz	

k_encoder_task_stack_sz	

Definition at line 401 of file main.c.

```
00401           : uint16_t {
00402     k_motor_task_stack_sz   = 2048U,
00403     k_encoder_task_stack_sz = 2048U,
00404 } rtos_stack_t;
```

4.7.4.7 status_led_pins_t

enum status_led_pins_t : uint8_t

Enumerator

k_port_7	
k_port_a	
k_port_b	
k_port_e	
k_port_6	
k_bit_led_heartbeat	PA7 – D9, toggled 10 Hz by encoder_task
k_bit_led_init_done	PB0 – D10, solid once main() finishes init
k_bit_led_motor_run	P71 – D11, toggles on every motor_task step
k_bit_led_enc_tick	P72 – D12, toggles per encoder sample
k_bit_led_motor_alive	PB1 – D13, proves motor_task reached loop body

Definition at line 113 of file main.c.

```
00113           : uint8_t {
00114     k_port_7   = 7,
00115     k_port_a   = 10,
00116     k_port_b   = 11,
00117     k_port_e   = 14,
00118     k_port_6   = 6,
00119
00120     k_bit_led_heartbeat = 7, /**< PA7 -- D9, toggled 10 Hz by encoder_task */
00121     k_bit_led_init_done = 0, /**< PB0 -- D10, solid once main() finishes init */
00122     k_bit_led_motor_run = 1, /**< P71 -- D11, toggles on every motor_task step */
00123     k_bit_led_enc_tick  = 2, /**< P72 -- D12, toggles per encoder sample */
00124     k_bit_led_motor_alive = 1, /**< PB1 -- D13, proves motor_task reached loop body */
00125 } status_led_pins_t;
```

4.7.4.8 sweep_const_t

enum sweep_const_t : uint8_t

Enumerator

k_sweep_step_count	-100, -90, ..., 0, ..., +100

Definition at line 388 of file main.c.

```
00388           : uint8_t {
00389     k_sweep_step_count = 21U, /**< -100, -90, ..., 0, ..., +100 */
00390 } sweep_const_t;
```

4.7.5 Function Documentation

4.7.5.1 busy_wait_ms()

void busy_wait_ms (
uint32_t ms) [static]

Definition at line 204 of file main.c.

```
{
    for (uint32_t m = 0; m < ms; m++) {
        for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
            __asm__ volatile("nop");
        }
    }
}
```

References k_iters_per_ms.

Referenced by motor_drv_gpio_init().

Here is the caller graph for this function:

4.7.5.2 clock_init()

void clock_init (
void) [extern]

Definition at line 81 of file clock.c.

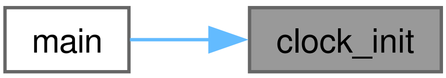
```
{
  volatile rx_system_regs_t *sys = system_regs();

  /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
  *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;

  /* Stop sub-clock oscillator -- not used on STAR. */
  sys->sosccr = k_sub_clock_stopped;

  /* Start MOSC (24 MHz external crystal). */
  sys->mosccr = k_main_osc_enabled;

  /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
   * settled (typically ~10 ms physical time) instead of waiting a fixed
   * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
  for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
    if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
      break;
    }
  }

  /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
  sys->pllcr  = k_pllcr_mosc_x10;
  sys->pllcr2 = k_pll_enabled;

  /* Wait for PLL lock (bounded). If we time out, the crystal is missing
   * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
   * on a dead clock. Stay on LOCO so the LEDs still respond and the user
   * can see we got past clock_init. */
  bool pll_locked = false;
  for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
    if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
      pll_locked = true;
      break;
    }
  }

```

```
  if (pll_locked) {
    /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
    *memwait_reg() = k_memwait_one_wait;

    /* Apply system clock dividers. */
    sys->sckcr = k_sckcr_value;

    /* Route UCLK from main PLL (UPLLSEL=0). */
    *((volatile uint16_t *)k_packcr_addr) = 0x0000U;

    /* Switch system clock source to PLL. */
    sys->sckcr3 = k_sckcr3_cksel_pll;
  }

  /* Re-lock PRCR. */
  *prcr_reg() = k_rx_prcr_lock;
}
```

References k_main_osc_enabled, k_main_osc_poll_max, k_oscovfsr_moovf, k_oscovfsr_plovf, k_packcr_addr, k_pll_enabled, k_pll_lock_poll_max, k_pllcr_mosc_x10, k_sckcr_value, and k_sub_clock_stopped.

Referenced by main().

Here is the caller graph for this function:



4.7.5.3 cmt0_init()

```
void cmt0_init (
        void )   [extern]
```

Definition at line 112 of file cmt0.c.

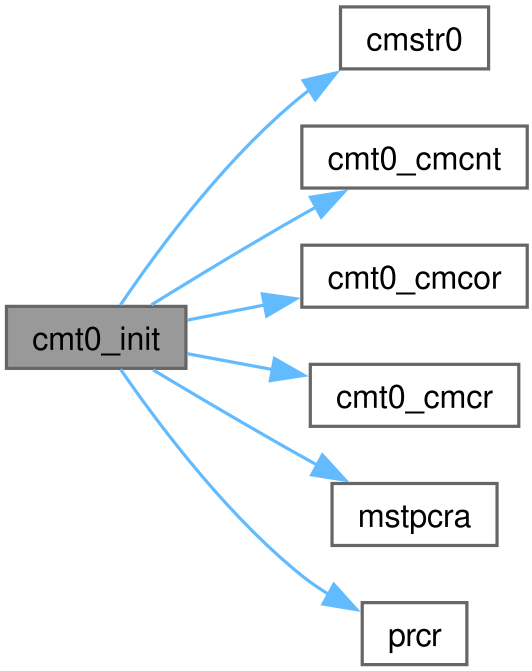
```
{
    /* Release CMT0 from module stop (MSTPCRA bit 15 controls the CMT0/CMT1 unit). */
    *prcr() = k_prcr_unlock_prc1;
    *mstpcra() &= ~(uint32_t)(1UL « (uint32_t)k_mstpcra_cmt0_bit);
    *prcr() = k_prcr_lock;

    /* Stop CMT0, configure, then start. */
    *cmstr0()    &= (uint16_t)~(uint16_t)k_cmstr0_ch0_bit;
    *cmt0_cmcr()  = (uint16_t)k_cmt0_cmcr_val;
    *cmt0_cmcor() = (uint16_t)k_cmt0_cmcor_val;
    *cmt0_cmcnt() = 0U;

    /* Direct-address writes -- bypass any macro/accessor doubt.  Addresses
     * verified against libs/rx_hal/inc/rx72n_icu_regs.h: IR/IER/IPR are
     * uint8_t arrays at offsets 0x000/0x200/0x300 from ICU base 0x00087000.
     *   IR[28]  = 0x0008_701C
     *   IER[3]  = 0x0008_7203, bit 4 = vector 28
     *   IPR[28] = 0x0008_731C
     * Priority raised from 3 to 10 to match production (rx72n_icu_regs.h
     * example doc). */
    /* ICU setup for CMT0_CMI0 (vector 28).  Direct-address writes document
     * each register explicitly; indices verified against
     * libs/rx_hal/inc/rx72n_icu_regs.h (@0x000 IR, @0x200 IER, @0x300 IPR).
     *
     * IMPORTANT -- RX72N IPR INDEX IS *NOT* THE VECTOR NUMBER.
     * The HAL's example code `icu_regs->ipr[28] = 10‘ at
     * rx72n_icu_regs.h:318 is WRONG for CMT0_CMI0.  IPR[] on RX72N is
     * sparse-mapped per a hardware lookup table in the RX72N HW manual
     * section 14 (ICU).  Vector 28 (CMT0_CMI0) maps to **IPR[4]**, not
     * IPR[28].  Writes to IPR[28] (0x8731C) are silently dropped -- reads
     * return 0 regardless of what was written.  Confirmed empirically:
     * writing 10 to IPR[4] made the ISR fire (g_cmt0_isr_count advanced
     * from 0 to 604+ in ~6 sec); writing the same to IPR[28] had zero
     * effect.  See also eclipse-threadx/rtos-docs and Renesas community
     * threads referenced in the matching RCA session notes.
     *
     * We intentionally keep BOTH writes for now: IPR[4] is the real
```

```
     * priority register; IPR[28] is a no-op we leave in so future readers
     * can trivially see the wrong-address trap that cost us a session,
     * with this comment serving as the "why".  Remove the IPR[28] write
     * after this test is known-stable on hardware. */
    *(volatile uint8_t *)0x0008701CU = 0U;   /* IR[28]  -- clear pending */
    *(volatile uint8_t *)0x00087304U = 10U;  /* IPR[4]  -- real priority register for vector 28 on RX72N */
    *(volatile uint8_t *)0x0008731CU = 10U;  /* IPR[28] -- NO-OP, intentionally kept as a tombstone (see comment above)
*/
    *(volatile uint8_t *)0x00087203U |= (uint8_t)(1U << 4); /* IER[3] bit 4 enable */

    *cmstr0() |= (uint16_t)k_cmstr0_ch0_bit;

    /* Globally enable interrupts -- ThreadX expects PSW.I = 1 before kernel entry. */
    __asm__ volatile("setpsw i");
}
```

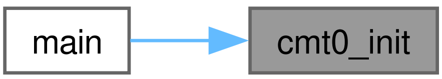
References cmstr0(), cmt0_cmcnt(), cmt0_cmcor(), cmt0_cmcr(), k_cmstr0_ch0_bit, k_cmt0_cmcor_val, k_cmt0_cmcr_val, k_mstpcra_cmt0_bit, k_prcr_lock, k_prcr_unlock_prc1, mstpcra(), and prcr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.4 encoder_read_raw()

```
uint16_t encoder_read_raw (
                uint8_t motor_idx)   [static]
```

Definition at line 335 of file main.c.

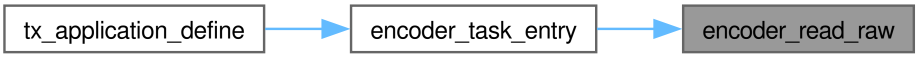
```
{
```

```
    /* IMPORTANT -- indices 2 and 3 are INTENTIONALLY SWAPPED relative to the
     * k_motors[] table's `.timer_channel` field and the previous "M2->TPU1,
     * M3->TPU2" comment.  Cross-check against motor_spin_test IPROPI on
     * 2026-04-21 showed M2 (BR) drawing ~0 current at every duty (dead
     * motor, hardware issue) while the encoder at TPU1 was counting fast
     * and the encoder at TPU2 was pinned at 0.  The only consistent reading
     * is that the physical encoder harness wires BR's encoder to TPU2
     * (TCLKC/TCLKD on PC0/PB3) and BL's encoder to TPU1 (TCLKA/TCLKB on
     * PC2/PA3) -- the opposite of what the k_motors[] comment claims.
     * Swapping the indices here makes m2 track BR and m3 track BL so the
     * CSV column matches the motor driver index in motor_spin_test. */
    static const uintptr_t k_tcnt_addr[k_motor_count] = {
        0x000C1386U,       /* MTU1 -- M0 (FL) */
        0x000C1406U,       /* MTU2 -- M1 (FR) */
        0x00088130U + 6U,  /* TPU2 -- M2 (BR): harness wires BR -> TPU2 */
        0x00088120U + 6U,  /* TPU1 -- M3 (BL): harness wires BL -> TPU1 */
    };
    return REG16_AT(k_tcnt_addr[motor_idx]);
}
```

References k_motor_count, and REG16_AT.

Referenced by encoder_task_entry().

Here is the caller graph for this function:



4.7.5.5 encoder_task_entry()

void encoder_task_entry (
ULONG arg) [static]

Definition at line 486 of file main.c.

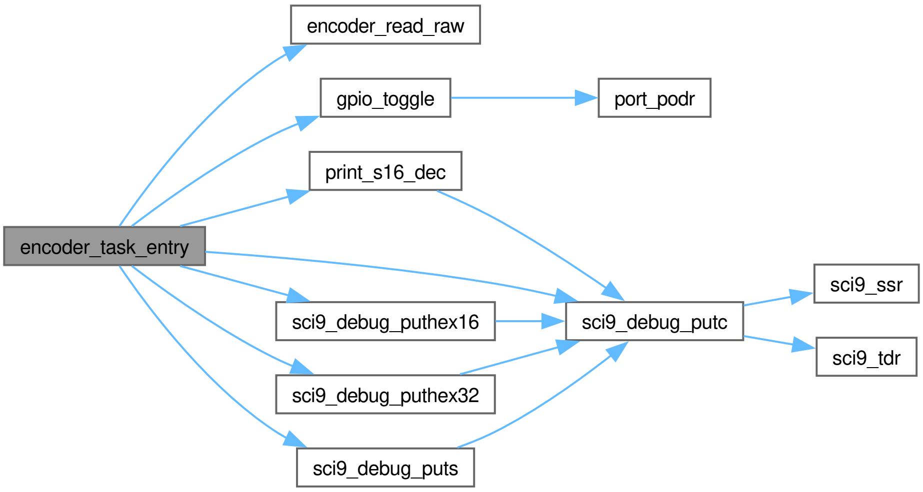
```
{
    (void)arg;

    sci9_debug_puts("t_ms,cmt0_cnt,mot_state,m_ent,duty,m0,m1,m2,m3\n");

    uint32_t t_ms = 0U;

    for (;;) {
        gpio_toggle(k_port_a, k_bit_led_heartbeat);
        gpio_toggle(k_port_7, k_bit_led_enc_tick);

        const uint16_t c0 = encoder_read_raw(0U);
        const uint16_t c1 = encoder_read_raw(1U);
        const uint16_t c2 = encoder_read_raw(2U);
        const uint16_t c3 = encoder_read_raw(3U);


        print_s16_dec((int16_t)(t_ms & 0x7FFFU));
        sci9_debug_putc(',');
        print_s16_dec((int16_t)(g_cmt0_isr_count & 0x7FFFU));
        sci9_debug_putc(',');
        print_s16_dec((int16_t)s_motor_tcb.tx_thread_state);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)g_motor_task_entries);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)s_current_duty);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)c0);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)c1);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)c2);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)c3);
        sci9_debug_putc('\n');

        /* Option 1b diagnostic: capture all TX_CALLER_ERROR discriminators
         * BEFORE sleep plus its return value AFTER.  ss and cur distinguish
         * the three remaining TX_CALLER_ERROR paths in tx_thread_sleep.c. */
```

```
        const UINT       pd  = _tx_thread_preempt_disable;
        const ULONG       ss  = _tx_thread_system_state;
        const TX_THREAD  *cur = _tx_thread_current_ptr;
        const UINT       ret = tx_thread_sleep((ULONG)k_encoder_sleep_ticks);
        sci9_debug_puts("pd=");
        print_s16_dec((int16_t)pd);
        sci9_debug_puts(" ss=");
        sci9_debug_puthex32((uint32_t)ss);
        sci9_debug_puts(" cur=");
        sci9_debug_puthex32((uint32_t)(uintptr_t)cur);
        sci9_debug_puts(" ret=");
        sci9_debug_puthex16((uint16_t)ret);
        sci9_debug_putc('\n');
        t_ms += 100U;
    }
}
```

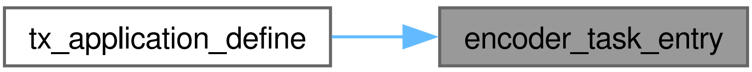
References _tx_thread_current_ptr, _tx_thread_preempt_disable, _tx_thread_system_state, encoder_read_raw(), g_cmt0_isr_count, g_motor_task_entries, gpio_toggle(), k_bit_led_enc_tick, k_bit_led_heartbeat, k_encoder_sleep_ticks, k_port_7, k_port_a, print_s16_dec(), s_current_duty, s_motor_tcb, sci9_debug_putc(), sci9_debug_puthex16(), sci9_debug_puthex32(), and sci9_debug_puts().

Referenced by tx_application_define().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.6 encoders_init()

bool encoders_init (
void) [static]

Definition at line 270 of file main.c.

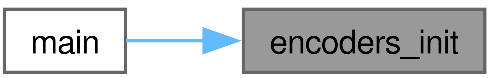
```
00271 {
00272     /* Step 1: release MTU (MSTPA9) + TPU (MSTPA13) module clocks. */
00273     *prcr_reg() = k_rx_prcr_unlock_all;
00274     system_regs()->mstpcra &= ~(uint32_t)((1UL « 9) | (uint32_t)k_tpu_mstpcra_mstpa13);
00275     *prcr_reg() = k_rx_prcr_lock;
00276
```

```
    /* Step 2 + 3: MPC pin sequence PMR=0, write PFS, PMR=1 for every
     * encoder phase input. */
    volatile uint8_t *pwpr  = (volatile uint8_t *)0x0008C11FU;
    volatile uint8_t *p2pmr = (volatile uint8_t *)0x0008C062U;
    volatile uint8_t *papmr = (volatile uint8_t *)0x0008C06AU;
    volatile uint8_t *pbpmr = (volatile uint8_t *)0x0008C06BU;
    volatile uint8_t *pcpmr = (volatile uint8_t *)0x0008C06CU;

    *p2pmr &= (uint8_t)~(uint8_t)((1U « 4) | (1U « 5));
    *papmr &= (uint8_t)~(uint8_t)((1U « 1) | (1U « 3));
    *pcpmr &= (uint8_t)~(uint8_t)((1U « 0) | (1U « 2) | (1U « 5));
    *pbpmr &= (uint8_t)~(uint8_t)(1U « 3);

    *pwpr = 0x00U;
    *pwpr = 0x40U;
    REG8_AT(0x0008C140U + 2U * 8U + 4U)  = 0x02U; /* P24 MTCLKA */
    REG8_AT(0x0008C140U + 2U * 8U + 5U)  = 0x02U; /* P25 MTCLKB */
    REG8_AT(0x0008C140U + 10U * 8U + 1U) = 0x02U; /* PA1 MTCLKC */
    REG8_AT(0x0008C140U + 12U * 8U + 5U) = 0x02U; /* PC5 MTCLKD */
    REG8_AT(0x0008C140U + 12U * 8U + 2U) = 0x03U; /* PC2 TCLKA  */
    REG8_AT(0x0008C140U + 10U * 8U + 3U) = 0x04U; /* PA3 TCLKB  */
    REG8_AT(0x0008C140U + 12U * 8U + 0U) = 0x03U; /* PC0 TCLKC  */
    REG8_AT(0x0008C140U + 11U * 8U + 3U) = 0x04U; /* PB3 TCLKD  */
    *pwpr = 0x00U;
    *pwpr = 0x80U;

    *p2pmr |= (uint8_t)((1U « 4) | (1U « 5));
    *papmr |= (uint8_t)((1U « 1) | (1U « 3));
    *pcpmr |= (uint8_t)((1U « 0) | (1U « 2) | (1U « 5));
    *pbpmr |= (uint8_t)(1U « 3);

    /* Phase counting mode 1 on MTU1 + MTU2 */
    volatile rx_mtu_channel_regs_t *m1 = mtu1();
    m1->tcr = 0;  m1->tmdr = 0x04U;
    m1->tiorh = 0; m1->tiorl = 0;
    m1->tier = 0;  m1->tsr = 0;  m1->tcnt = 0;
    volatile rx_mtu_channel_regs_t *m2 = mtu2();
    m2->tcr = 0;  m2->tmdr = 0x04U;
    m2->tiorh = 0; m2->tiorl = 0;
    m2->tier = 0;  m2->tsr = 0;  m2->tcnt = 0;

    /* Phase counting mode 1 on TPU1 + TPU2 */
    volatile rx_tpu_regs_t *t1 = tpu1();
    t1->tcr = 0;  t1->tmdr = 0x04U;  t1->tcnt = 0;
    volatile rx_tpu_regs_t *t2 = tpu2();
    t2->tcr = 0;  t2->tmdr = 0x04U;  t2->tcnt = 0;

    /* Clear TPU noise filters from any prior-run leftover state */
    tpu_control()->nfcr[1] = 0;
    tpu_control()->nfcr[2] = 0;

    /* Start all four counters */
    mtu_tstra()->tstr   |= (uint8_t)(k_mtu_tstr_cst1 | k_mtu_tstr_cst2);
    tpu_control()->tstr |= (uint8_t)(k_tpu_tstr_cst1 | k_tpu_tstr_cst2);

    return true;
}
```

References REG8_AT.

Referenced by main().

Here is the caller graph for this function:



4.7.5.7 error_park()

void error_park (
const char ∗ msg) [static]

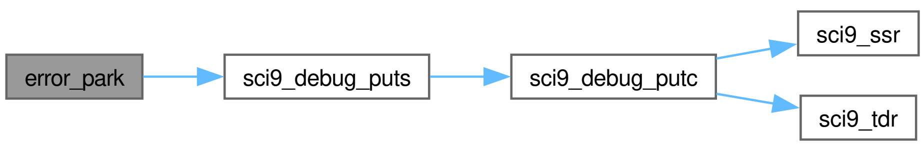
Definition at line 568 of file main.c.

```
{
    sci9_debug_puts("FATAL ");
    sci9_debug_puts(msg);
    sci9_debug_puts("\n");

    /* Best-effort motor safe: attempt a 0-duty write (may fail if rx_motor
     * isn't initialised, but that's OK since GPTW would be idle in that case). */
    for (uint8_t i = 0; i < k_motor_count; i++) {
        (void)rx_motor_set_duty(&s_motors[i], 0.0F);
    }

    for (;;) {
        __asm__ volatile("nop");
    }
}
```

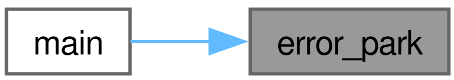
References k_motor_count, s_motors, and sci9_debug_puts().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.8 gpio_set_output()

void gpio_set_output (
    uint8_t port,
    uint8_t pin) [inline], [static]

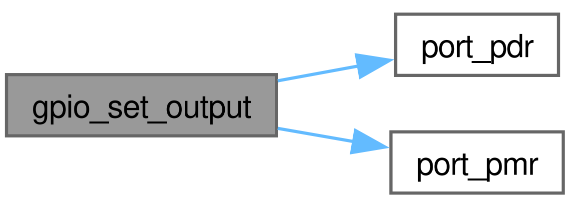
Definition at line 92 of file main.c.

```
00093 {
00094     *port_pmr(port) &= (uint8_t) ~(1U << pin);
00095     *port_pdr(port) |= (uint8_t)(1U << pin);
00096 }
```

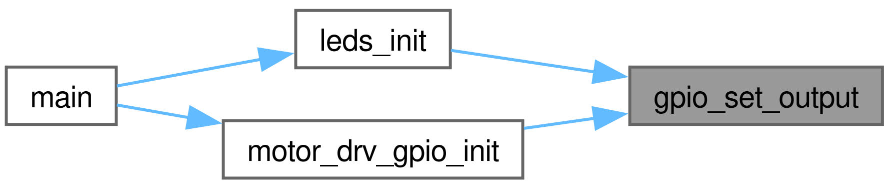
References port_pdr(), and port_pmr().

Referenced by leds_init(), and motor_drv_gpio_init().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.9 gpio_toggle()

void gpio_toggle (
    uint8_t port,
    uint8_t pin) [inline], [static]

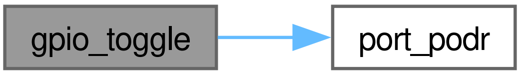
Definition at line 105 of file main.c.

```
{
    *port_podr(port) ^= (uint8_t)(1U « pin);
}
```

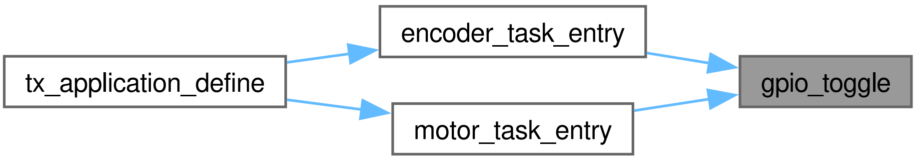
References port_podr().

Referenced by encoder_task_entry(), and motor_task_entry().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.10 gpio_write()

void gpio_write (
    uint8_t port,
    uint8_t pin,
    bool high) [inline], [static]

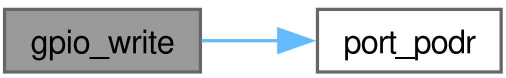
Definition at line 97 of file main.c.

```
{
    if (high) {
        *port_podr(port) |= (uint8_t)(1U « pin);
    } else {
        *port_podr(port) &= (uint8_t) ~(1U « pin);
    }
}
```

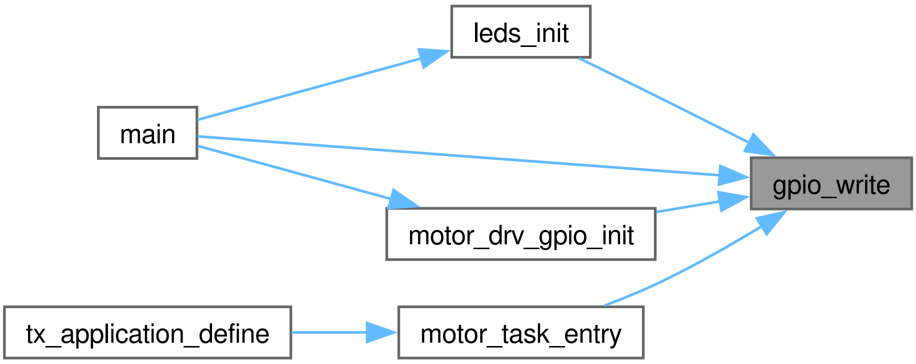
References port_podr().

Referenced by leds_init(), main(), motor_drv_gpio_init(), and motor_task_entry().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.11 leds_init()

void leds_init (
void) [static]

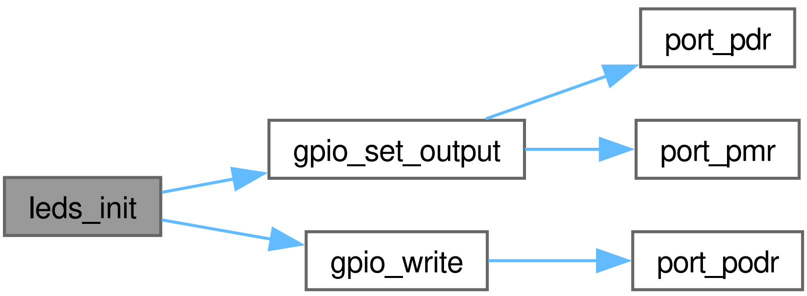
Definition at line 127 of file main.c.

```
{
    gpio_set_output(k_port_a, k_bit_led_heartbeat);
    gpio_write(k_port_a, k_bit_led_heartbeat, false);

    gpio_set_output(k_port_b, k_bit_led_init_done);
    gpio_write(k_port_b, k_bit_led_init_done, false);

    gpio_set_output(k_port_7, k_bit_led_motor_run);
    gpio_write(k_port_7, k_bit_led_motor_run, false);

    gpio_set_output(k_port_7, k_bit_led_enc_tick);
    gpio_write(k_port_7, k_bit_led_enc_tick, false);

    gpio_set_output(k_port_b, k_bit_led_motor_alive);
    gpio_write(k_port_b, k_bit_led_motor_alive, false);
}
```

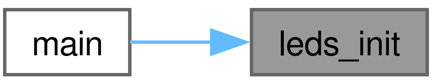
References gpio_set_output(), gpio_write(), k_bit_led_enc_tick, k_bit_led_heartbeat, k_bit_led_init_done, k_bit_led_motor_alive, k_bit_led_motor_run, k_port_7, k_port_a, and k_port_b.

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.12 main()

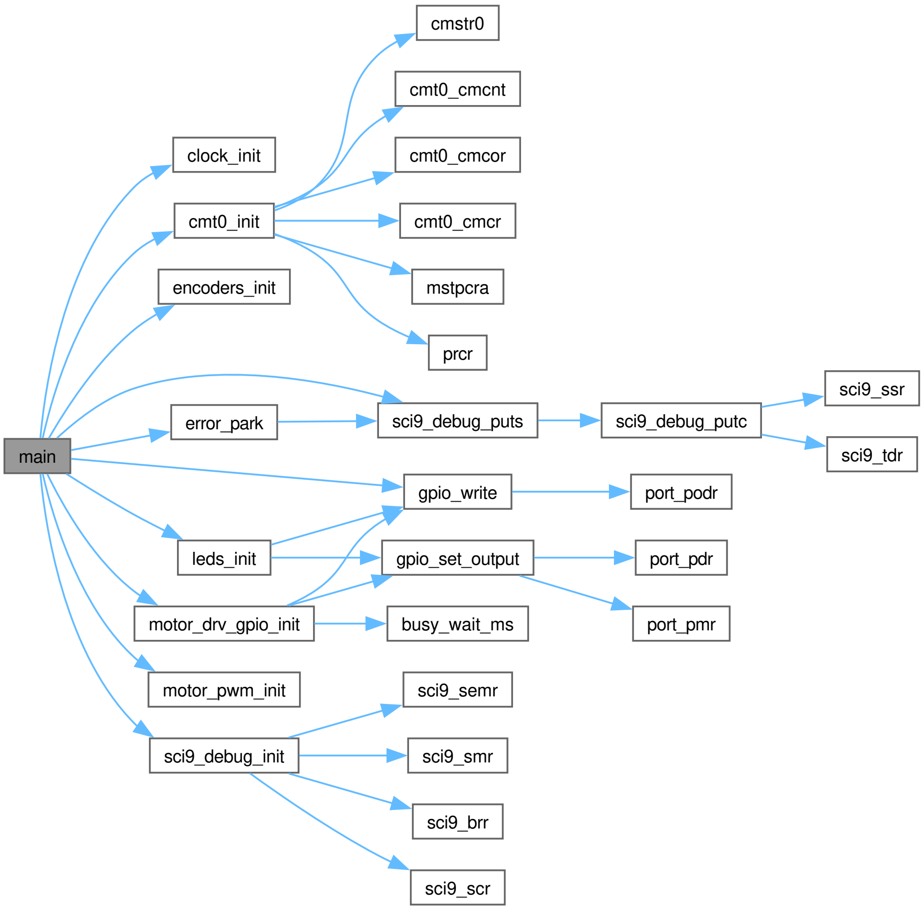
```
int main (
            void )
```

Definition at line 588 of file main.c.

```c
{
    clock_init();
    leds_init();
    sci9_debug_init();

    sci9_debug_puts("\n\n");
    sci9_debug_puts("motors_rtos: ThreadX + rx_motor + rx_encoder bring-up\n");
    sci9_debug_puts("clock=240MHz pclkb=60MHz tick=100Hz\n");

    motor_drv_gpio_init();
    sci9_debug_puts("drv8263 armed\n");

    if (!motor_pwm_init(s_motors)) {
        error_park("motor_pwm_init");
    }
    sci9_debug_puts("motors 0..3 init ok\n");

    if (!encoders_init()) {
        error_park("encoders_init");
    }
    sci9_debug_puts("encoders 0..3 init ok\n");

    gpio_write(k_port_b, k_bit_led_init_done, true);
    sci9_debug_puts("entering scheduler\n");

    cmt0_init();
    tx_kernel_enter();

    /* tx_kernel_enter() never returns; fall-through trap if it somehow does. */
    for (;;) {
        __asm__ volatile("nop");
    }
    return 0;
}
```

References clock_init(), cmt0_init(), encoders_init(), error_park(), gpio_write(), k_bit_led_init_done, k_port_b, leds_init(), motor_drv_gpio_init(), motor_pwm_init(), s_motors, sci9_debug_init(), and sci9_debug_puts().

Here is the call graph for this function:



4.7.5.13 motor_drv_gpio_init()

void motor_drv_gpio_init (
void) [static]

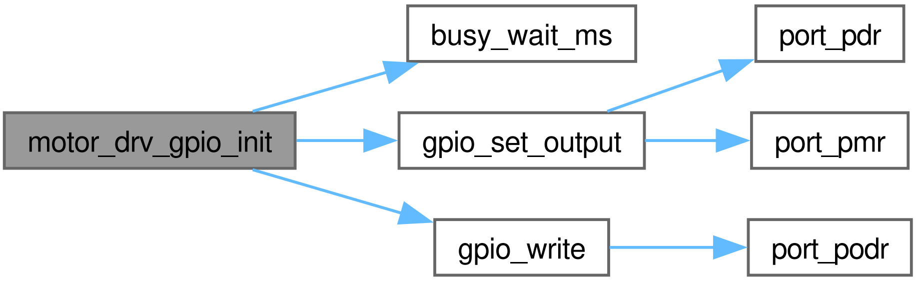
Definition at line 216 of file main.c.

```
{
    for (uint8_t i = 0; i < k_motor_count; i++) {
        gpio_set_output(k_motors[i].drvoff_port, k_motors[i].drvoff_pin);
        gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, true);
    }
    for (uint8_t i = 0; i < k_motor_count; i++) {
        gpio_set_output(k_motors[i].nsleep_port, k_motors[i].nsleep_pin);
        gpio_write(k_motors[i].nsleep_port, k_motors[i].nsleep_pin, true);
    }
    busy_wait_ms(10);
    for (uint8_t i = 0; i < k_motor_count; i++) {
        gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, false);
    }
}
```

References busy_wait_ms(), gpio_set_output(), gpio_write(), k_motor_count, and k_motors.

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:

4.7.5.14 motor_pwm_init()

bool motor_pwm_init (
    rx_motor_handle_t handles[k_motor_count]) [static]

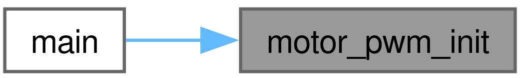
Definition at line 240 of file main.c.

```
{
    for (uint8_t i = 0; i < k_motor_count; i++) {
        const rx_motor_config_t cfg = {
            .channel       = k_motors[i].gptw_channel,
            .output_a      = k_gptw_output_a,
            .output_b      = k_gptw_output_b,
            .pwm_freq_hz   = k_pwm_freq_hz,
            .dead_time_ns = k_deadtime_ns,
            .invert_pwm    = false,
            .port_a_idx    = (uint8_t)rx_port_from_pin(k_motors[i].in2_pin),
            .bit_a         = (uint8_t)rx_pin_from_pin(k_motors[i].in2_pin),
            .port_b_idx    = (uint8_t)rx_port_from_pin(k_motors[i].in1_pin),
            .bit_b         = (uint8_t)rx_pin_from_pin(k_motors[i].in1_pin),
        };
        if (rx_motor_init(&handles[i], &cfg) != k_rx_ok) {
            return false;
        }
    }
    return true;
}
```

References k_deadtime_ns, k_motor_count, k_motors, and k_pwm_freq_hz.

Referenced by main().

Here is the caller graph for this function:



4.7.5.15 motor_task_entry()

void motor_task_entry (
ULONG arg) [static]

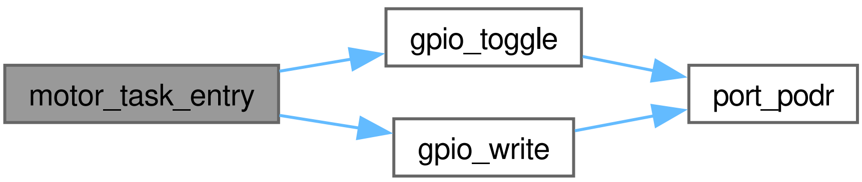
Definition at line 448 of file main.c.

```
{
    (void)arg;

    g_motor_task_entries = 1U;
    gpio_write(k_port_b, k_bit_led_motor_alive, true);

    /* Explicit 0 duty on entry for a clean baseline before the sweep. */
    for (uint8_t i = 0; i < k_motor_count; i++) {
        (void)rx_motor_set_duty(&s_motors[i], 0.0F);
    }

    uint8_t step = 0U;

    for (;;) {
        gpio_toggle(k_port_7, k_bit_led_motor_run);

        const int8_t duty = k_sweep_table[step];
        s_current_duty = duty;

        for (uint8_t i = 0; i < k_motor_count; i++) {
            const float signed_duty = (float)(int16_t)duty *
                                      (float)(int16_t)k_motors[i].direction_sign;
            (void)rx_motor_set_duty(&s_motors[i], signed_duty);
        }

        step = (uint8_t)((step + 1U) % k_sweep_step_count);

        (void)tx_thread_sleep((ULONG)k_motor_sleep_ticks);
    }
}
```

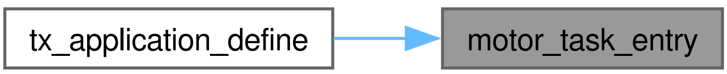
References g_motor_task_entries, gpio_toggle(), gpio_write(), k_bit_led_motor_alive, k_bit_led_motor_run, k_motor_count, k_motor_sleep_ticks, k_motors, k_port_7, k_port_b, k_sweep_step_count, k_sweep_table, s_current_duty, and s_motors.

Referenced by tx_application_define().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.16 port_pdr()

volatile uint8_t * port_pdr (
    uint8_t port) [inline], [static]

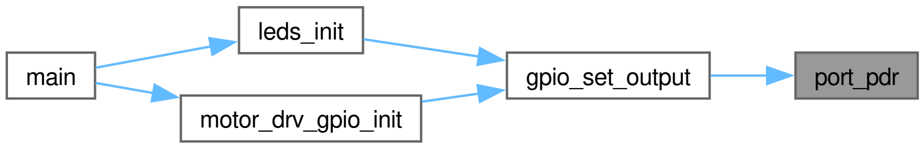
Definition at line 79 of file main.c.

```
00080 {
00081     return &REG8(k_port_pdr_base + port);
00082 }
```

References k_port_pdr_base, and REG8.

Referenced by gpio_set_output().

Here is the caller graph for this function:



4.7.5.17 port_pmr()

volatile uint8_t * port_pmr (
    uint8_t port) [inline], [static]

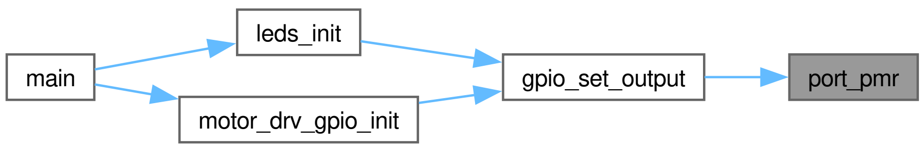
Definition at line 87 of file main.c.

```
00088 {
00089     return &REG8(k_port_pmr_base + port);
00090 }
```

References k_port_pmr_base, and REG8.

Referenced by gpio_set_output().

Here is the caller graph for this function:



4.7.5.18 port_podr()

volatile uint8_t ∗ port_podr (
uint8_t port) [inline], [static]

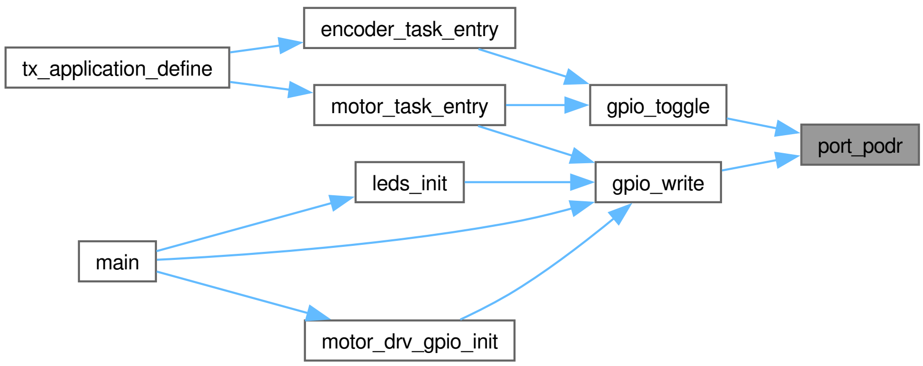
Definition at line 83 of file main.c.

```
{
    return &REG8(k_port_podr_base + port);
}
```

References k_port_podr_base, and REG8.

Referenced by gpio_toggle(), and gpio_write().

Here is the caller graph for this function:



4.7.5.19 print_s16_dec()

void print_s16_dec (
int16_t v) [static]

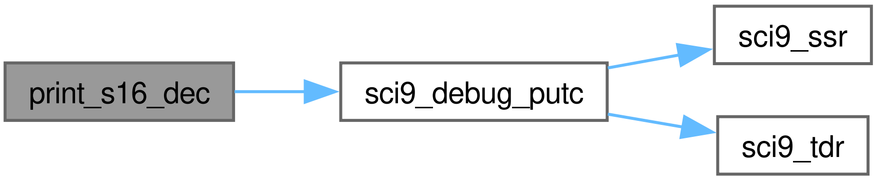
Definition at line 360 of file main.c.

```
{
    char buf[8];
    uint8_t  i        = 0;
    uint32_t u;
    bool     negative = v < 0;

    u = negative ? (uint32_t)(-(int32_t)v) : (uint32_t)v;
    if (u == 0U) {
        sci9_debug_putc('0');
        return;
    }
    if (negative) {
        sci9_debug_putc('-');
    }
    while (u != 0U) {
        buf[i++] = (char)('0' + (u % 10U));
        u /= 10U;
    }
    while (i-- > 0U) {
        sci9_debug_putc(buf[i]);
    }
}
```

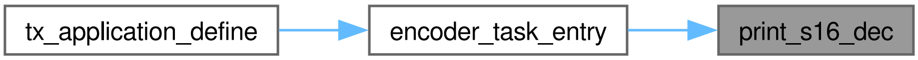
References sci9_debug_putc().

Referenced by encoder_task_entry().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.20 sci9_debug_init()

void sci9_debug_init (
    void) [extern]

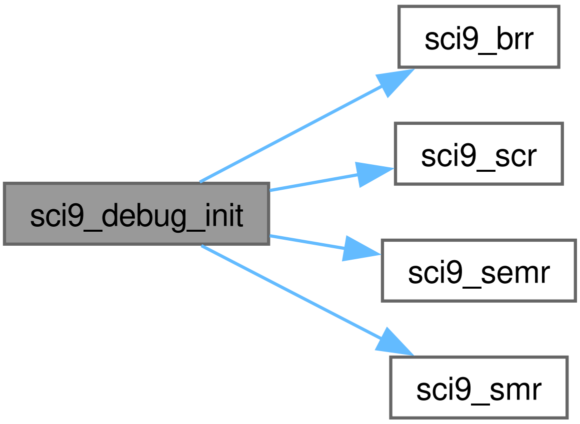
Definition at line 104 of file sci9.c.

```
{
    /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
     * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
    *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
    system_regs()->mstpcrc &= ~(uint32_t)(1UL << (uint32_t)k_mstpc_sci9_bit);
    *prcr_reg() = k_rx_prcr_lock;

    /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
     * accessor so the compiler computes the correct PFS offsets via
     * static_assert-checked layout. */
    mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
    mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
    mpc()->pb6pfs = k_psel_sci9;
    mpc()->pb7pfs = k_psel_sci9;
    mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
    mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */

    /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
     * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
    portb()->pdr |= (uint8_t)k_pb7_mask;
    portb()->pdr &= (uint8_t) ~k_pb6_mask;
    portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);

    /* Step 4: SCI9 init.  Match the production order:
     *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
     * Don't touch SCMR (reset default 0xF2 is correct). */
    *sci9_scr() = k_scr_disabled;
    *sci9_smr() = k_smr_8n1_async;
    *sci9_brr() = k_brr_115200;

    /* Wait roughly one bit time before enabling TX/RX. */
    for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
        __asm__ volatile("nop");
    }

    *sci9_scr()  = k_scr_txrx_enabled;
    *sci9_semr() = k_semr_default;
}
```

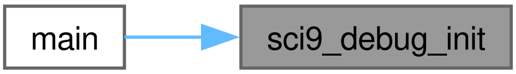
References k_brr_115200, k_brr_settle_loops, k_mstpc_sci9_bit, k_pb6_mask, k_pb7_mask, k_psel_sci9, k_pwpr_b0wi, k_pwpr_pfswe, k_pwpr_unlock_b0, k_scr_disabled, k_scr_txrx_enabled, k_semr_default, k_smr_8n1_async, sci9_brr(), sci9_scr(), sci9_semr(), and sci9_smr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.21 sci9_debug_putc()

void sci9_debug_putc (
char c) [extern]

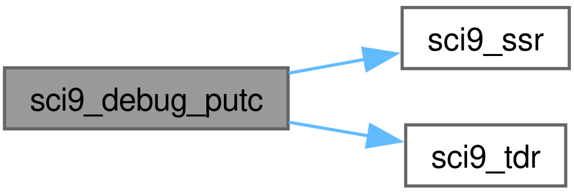
Definition at line 147 of file sci9.c.

```
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
```

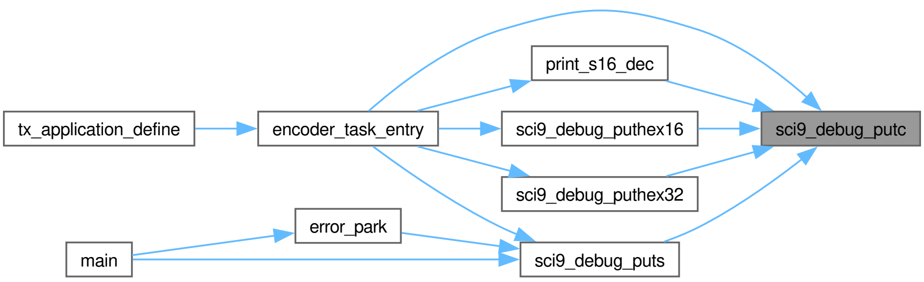
References k_ssr_tdre, sci9_ssr(), and sci9_tdr().

Referenced by encoder_task_entry(), print_s16_dec(), sci9_debug_puthex16(), sci9_debug_puthex32(), and sci9_debug_puts().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.22 sci9_debug_puthex16()

void sci9_debug_puthex16 (
uint16_t v) [extern]

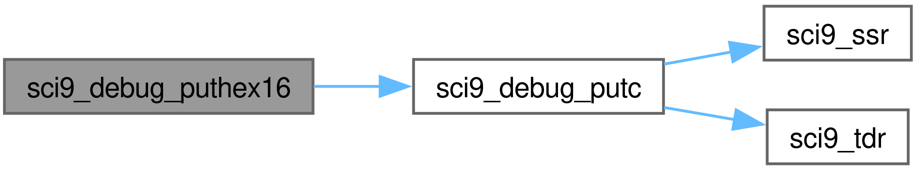
Definition at line 179 of file sci9.c.

```
{
    static const char hex_digits[] = "0123456789ABCDEF";
    sci9_debug_putc('0');
    sci9_debug_putc('x');
    for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
        shift -= (int8_t)k_hex_shift_step) {
       sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
    }
}
```

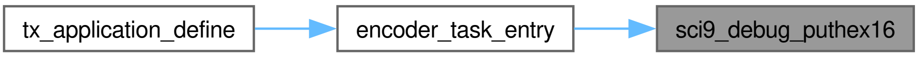
References k_hex_nibble_mask, k_hex_shift_init16, k_hex_shift_step, and sci9_debug_putc().

Referenced by encoder_task_entry().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.23 sci9_debug_puthex32()

void sci9_debug_puthex32 (
uint32_t v) [extern]

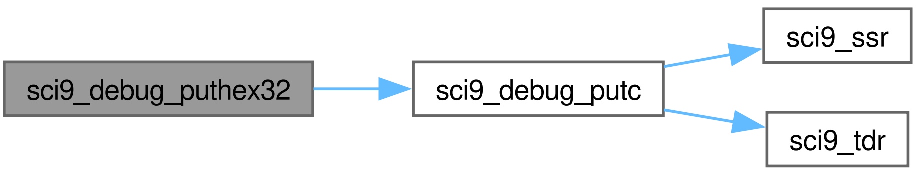
Definition at line 168 of file sci9.c.

```
{
    static const char hex_digits[] = "0123456789ABCDEF";
    sci9_debug_putc('0');
    sci9_debug_putc('x');
    for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
        shift -= (int8_t)k_hex_shift_step) {
       sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
    }
}
```

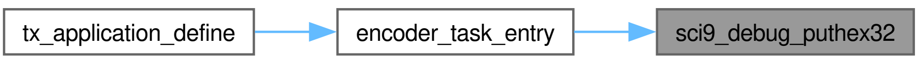
References k_hex_nibble_mask, k_hex_shift_init32, k_hex_shift_step, and sci9_debug_putc().

Referenced by encoder_task_entry().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.24 sci9_debug_puts()

void sci9_debug_puts (
const char * s) [extern]

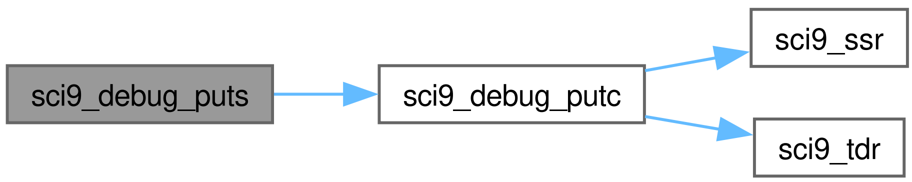
Definition at line 155 of file sci9.c.

```
{
    if (s == (const char *)0) {
        return;
    }
    while (*s != '\0') {
        if (*s == '\n') {
            sci9_debug_putc('\r');
        }
        sci9_debug_putc(*s++);
    }
}
```

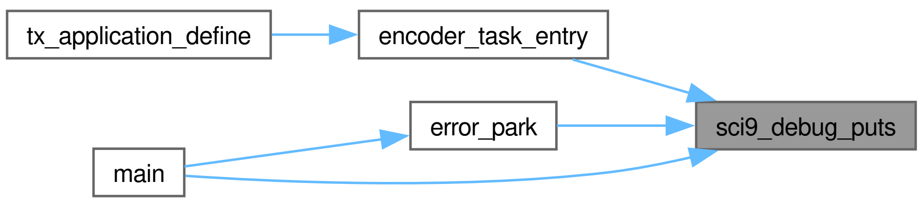
References sci9_debug_putc().

Referenced by encoder_task_entry(), error_park(), and main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.5.25 tx_application_define()

void tx_application_define (
void * first_unused_memory)

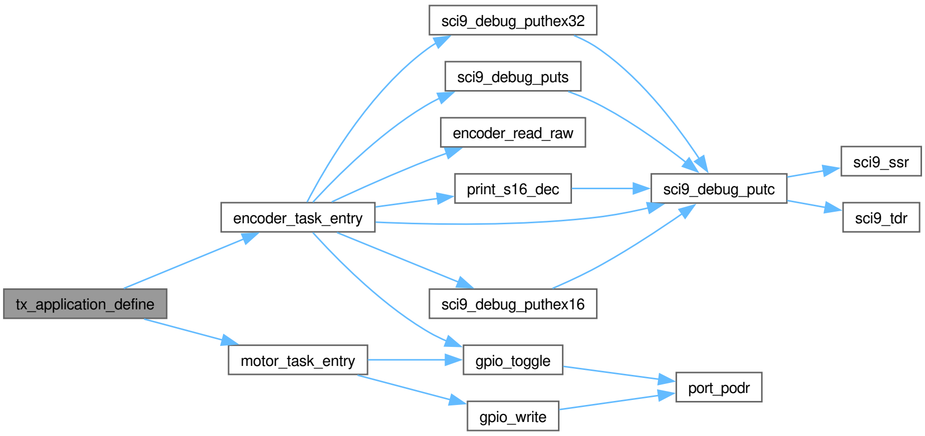
Definition at line 546 of file main.c.

```
{
    (void)first_unused_memory;

    g_encoder_create_status = (uint32_t)tx_thread_create(
        &s_encoder_tcb, "encoder", encoder_task_entry, 0U,
        s_encoder_stack, (ULONG)k_encoder_task_stack_sz,
        (UINT)k_encoder_task_priority,
        (UINT)k_encoder_task_priority,
        TX_NO_TIME_SLICE, TX_AUTO_START);

    g_motor_create_status = (uint32_t)tx_thread_create(
        &s_motor_tcb, "motor", motor_task_entry, 0U,
        s_motor_stack, (ULONG)k_motor_task_stack_sz,
        (UINT)k_motor_task_priority,
        (UINT)k_motor_task_priority,
```

```
00562         TX_NO_TIME_SLICE, TX_AUTO_START);
00563 }
```

References encoder_task_entry(), g_encoder_create_status, g_motor_create_status, k_encoder_task_priority, k_encoder_task_stack_sz, k_motor_task_priority, k_motor_task_stack_sz, motor_task_entry(), s_encoder_stack, s_encoder_tcb, s_motor_stack, and s_motor_tcb.

Here is the call graph for this function:



4.7.6 Variable Documentation

4.7.6.1 _tx_thread_current_ptr

TX_THREAD∗ _tx_thread_current_ptr [extern]

Referenced by encoder_task_entry().

4.7.6.2 _tx_thread_preempt_disable

UINT _tx_thread_preempt_disable [extern]

Referenced by encoder_task_entry().

4.7.6.3 _tx_thread_system_state

ULONG _tx_thread_system_state [extern]

Referenced by encoder_task_entry().

4.7.6.4 g_cmt0_isr_count

volatile uint32_t g_cmt0_isr_count [extern]

from cmt0.c – verifies CMT0 tick ISR is firing

Definition at line 107 of file cmt0.c.

Referenced by encoder_task_entry().

4.7.6.5 g_encoder_create_status

volatile uint32_t g_encoder_create_status = 0xFFFFFFFFU [static]

tx_thread_create return for encoder_task

Definition at line 431 of file main.c.

Referenced by tx_application_define().

4.7.6.6 g_motor_create_status

volatile uint32_t g_motor_create_status = 0xFFFFFFFFU [static]

tx_thread_create return for motor_task

Definition at line 430 of file main.c.

Referenced by tx_application_define().

4.7.6.7 g_motor_task_entries

volatile uint32_t g_motor_task_entries = 0U [static]

bumped once on task entry

Definition at line 427 of file main.c.

Referenced by encoder_task_entry(), and motor_task_entry().

4.7.6.8 g_motor_task_loops

volatile uint32_t g_motor_task_loops = 0U [static]

bumped each iteration BEFORE set_duty

Definition at line 428 of file main.c.

4.7.6.9 g_motor_task_loops_after

volatile uint32_t g_motor_task_loops_after = 0U [static]

bumped each iteration AFTER set_duty

Definition at line 429 of file main.c.

4.7.6.10 k_motors

const motor_wiring_t k_motors[k_motor_count] [static]

Initial value:

```
= {
{.gptw_channel = k_gptw_channel_0, .in2_pin = k_rx_p2_3, .in1_pin = k_rx_p1_7,
 .drvoff_port = k_port_6, .drvoff_pin = 1, .nsleep_port = k_port_6, .nsleep_pin = 0,
 .is_tpu = false, .timer_channel = 1, .direction_sign = -1},

{.gptw_channel = k_gptw_channel_1, .in2_pin = k_rx_p2_2, .in1_pin = k_rx_pc_3,
 .drvoff_port = k_port_6, .drvoff_pin = 3, .nsleep_port = k_port_6, .nsleep_pin = 2,
 .is_tpu = false, .timer_channel = 2, .direction_sign = +1},

{.gptw_channel = k_gptw_channel_2, .in2_pin = k_rx_pe_3, .in1_pin = k_rx_p8_6,
 .drvoff_port = k_port_e, .drvoff_pin = 0, .nsleep_port = k_port_6, .nsleep_pin = 4,
 .is_tpu = true,  .timer_channel = 1, .direction_sign = +1},

{.gptw_channel = k_gptw_channel_3, .in2_pin = k_rx_pe_7, .in1_pin = k_rx_pc_6,
 .drvoff_port = k_port_e, .drvoff_pin = 2, .nsleep_port = k_port_e, .nsleep_pin = 1,
 .is_tpu = true,  .timer_channel = 2, .direction_sign = -1},
}
```

Definition at line 176 of file main.c.

```
00176                                  {
00177     /* M0 = FL, wired backwards */
00178     {.gptw_channel = k_gptw_channel_0, .in2_pin = k_rx_p2_3, .in1_pin = k_rx_p1_7,
00179      .drvoff_port = k_port_6, .drvoff_pin = 1, .nsleep_port = k_port_6, .nsleep_pin = 0,
00180      .is_tpu = false, .timer_channel = 1, .direction_sign = -1},
00181     /* M1 = FR */
00182     {.gptw_channel = k_gptw_channel_1, .in2_pin = k_rx_p2_2, .in1_pin = k_rx_pc_3,
00183      .drvoff_port = k_port_6, .drvoff_pin = 3, .nsleep_port = k_port_6, .nsleep_pin = 2,
00184      .is_tpu = false, .timer_channel = 2, .direction_sign = +1},
00185     /* M2 = BR */
00186     {.gptw_channel = k_gptw_channel_2, .in2_pin = k_rx_pe_3, .in1_pin = k_rx_p8_6,
00187      .drvoff_port = k_port_e, .drvoff_pin = 0, .nsleep_port = k_port_6, .nsleep_pin = 4,
00188      .is_tpu = true,  .timer_channel = 1, .direction_sign = +1},
00189     /* M3 = BL, wired backwards */
00190     {.gptw_channel = k_gptw_channel_3, .in2_pin = k_rx_pe_7, .in1_pin = k_rx_pc_6,
00191      .drvoff_port = k_port_e, .drvoff_pin = 2, .nsleep_port = k_port_e, .nsleep_pin = 1,
00192      .is_tpu = true,  .timer_channel = 2, .direction_sign = -1},
00193 };
```

Referenced by motor_drv_gpio_init(), motor_pwm_init(), and motor_task_entry().

4.7.6.11 k_sweep_table

const int8_t k_sweep_table[k_sweep_step_count] [static]

Initial value:

```
= {
-100, -90, -80, -70, -60, -50, -40, -30, -20, -10,
   0,
 +10, +20, +30, +40, +50, +60, +70, +80, +90, +100,
}
```

Definition at line 392 of file main.c.

```
00392                                              {
00393     -100, -90, -80, -70, -60, -50, -40, -30, -20, -10,
00394        0,
00395      +10, +20, +30, +40, +50, +60, +70, +80, +90, +100,
00396 };
```

Referenced by motor_task_entry().

4.7.6.12 s_current_duty

volatile int8_t s_current_duty = 0 [static]

Definition at line 423 of file main.c.

Referenced by encoder_task_entry(), and motor_task_entry().

4.7.6.13 s_encoder_stack

uint8_t s_encoder_stack[k_encoder_task_stack_sz] [static]

Definition at line 416 of file main.c.

Referenced by tx_application_define().

4.7.6.14 s_encoder_tcb

TX_THREAD s_encoder_tcb [static]

Definition at line 414 of file main.c.

Referenced by tx_application_define().

4.7.6.15 s_motor_stack

uint8_t s_motor_stack[k_motor_task_stack_sz] [static]

Definition at line 415 of file main.c.

Referenced by tx_application_define().

4.7.6.16 s_motor_tcb

TX_THREAD s_motor_tcb [static]

Definition at line 413 of file main.c.

Referenced by encoder_task_entry(), and tx_application_define().

4.7.6.17 s_motors

rx_motor_handle_t s_motors[k_motor_count] [static]

Definition at line 419 of file main.c.

Referenced by error_park(), main(), and motor_task_entry().

4.8 main.c

Go to the documentation of this file.

```c
/**
 * @file main.c
 * @brief motors_rtos -- motors + encoders under Azure RTOS ThreadX.
 *
 * @details
 * Two-task ThreadX bring-up of libs/rx_motor + libs/rx_encoder, proving they
 * coexist with the scheduler.  Mirrors encoder_test's open-loop spin + count
 * logic but splits the work across two cooperating tasks:
 *
 *   motor_task    (priority 10) -- every 500 ms, advances a duty-sweep state
 *                                  machine.  All 4 motors take the same
 *                                  duty, with per-motor direction_sign applied
 *                                  so FL/BL spin the same wheel direction as
 *                                  FR/BR despite the reversed wiring.
 *
 *   encoder_task  (priority  9) -- every 100 ms, reads all 4 encoder TCNTs,
 *                                  prints a one-line CSV over SCI9 (115200,
 *                                  /dev/ttyACM0 on the host), and toggles the
 *                                  PA7/D9 heartbeat LED.  Higher priority
 *                                  than motor_task so the console stays
 *                                  responsive even when motor_task is awake.
 *
 * Boot sequence (main):
 *   1. clock_init()          -- PLL 240 MHz / PCLKA 120 / PCLKB 60
 *   2. sci9_debug_init()     -- console up; banner printed immediately
 *   3. motor_drv_gpio_init   -- DRVOFF high, nSLEEP high, tWAKE 10 ms, DRVOFF low
 *   4. motor_pwm_init        -- rx_motor_init x4 at 20 kHz, 1 us deadtime
 *   5. encoders_init         -- MTU1/MTU2/TPU1/TPU2 phase counting mode 1
 *   6. cmt0_init             -- 100 Hz ThreadX tick + setpsw i
 *   7. tx_kernel_enter       -- hands off forever
 *
 * Per memory project_motor_pwm_hal_bugs.md: the production rx_mpc / rx_gptw
 * stack has latent issues writing PFS for MTCLK/TCLK encoder pins, so the
 * encoders_init() routine here mirrors encoder_test's direct-register workaround
 * rather than calling rx_mpc_set_mtu_encoder() / rx_mpc_set_tpu_encoder().
 *
 * SPDX-License-Identifier: MIT
 * @copyright Copyright (c) 2026 Locked Inc.
 */

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#include "tx_api.h"

#include "rx_motor.h"
#include "rx_gptw.h"
#include "rx_mpc.h"
#include "rx_port_constants.h"
#include "rx_mtu_encoder.h"
#include "rx_encoder_tpu.h"
#include "rx72n_mtu_regs.h"
#include "rx72n_tpu_regs.h"
#include "rx72n_system_regs.h"
#include "rx72n_icu_regs.h"
#include "rx_register_protection.h"

/* Symbols defined in sibling files */
extern void clock_init(void);
extern void cmt0_init(void);
extern void sci9_debug_init(void);
extern void sci9_debug_putc(char c);
extern void sci9_debug_puts(const char *s);
extern void sci9_debug_puthex16(uint16_t v);
extern void sci9_debug_puthex32(uint32_t v);

/*
=============================================================================
 * Direct port-register access helpers (same pattern as encoder_test).
 *
=============================================================================
*/
#define REG8(a) (*(volatile uint8_t *)(uintptr_t)(a))

typedef enum : uintptr_t {
    k_port_pdr_base  = 0x0008C000U,
    k_port_podr_base = 0x0008C020U,
    k_port_pmr_base  = 0x0008C060U,
} port_reg_base_t;

static inline volatile uint8_t *port_pdr(uint8_t port)
```

```
{
    return &REG8(k_port_pdr_base + port);
}
static inline volatile uint8_t *port_podr(uint8_t port)
{
    return &REG8(k_port_podr_base + port);
}
static inline volatile uint8_t *port_pmr(uint8_t port)
{
    return &REG8(k_port_pmr_base + port);
}

static inline void gpio_set_output(uint8_t port, uint8_t pin)
{
    *port_pmr(port) &= (uint8_t) ~(1U « pin);
    *port_pdr(port) |= (uint8_t)(1U « pin);
}
static inline void gpio_write(uint8_t port, uint8_t pin, bool high)
{
    if (high) {
        *port_podr(port) |= (uint8_t)(1U « pin);
    } else {
        *port_podr(port) &= (uint8_t) ~(1U « pin);
    }
}
static inline void gpio_toggle(uint8_t port, uint8_t pin)
{
    *port_podr(port) ^= (uint8_t)(1U « pin);
}

/*
 ==========================================================================
 * Status LED map (matches encoder_test, matches memory project_star_pcb_leds.md)
 *
 ==========================================================================
 */
typedef enum : uint8_t {
    k_port_7   = 7,
    k_port_a   = 10,
    k_port_b   = 11,
    k_port_e   = 14,
    k_port_6   = 6,

    k_bit_led_heartbeat = 7, /**< PA7 -- D9, toggled 10 Hz by encoder_task */
    k_bit_led_init_done = 0, /**< PB0 -- D10, solid once main() finishes init */
    k_bit_led_motor_run = 1, /**< P71 -- D11, toggles on every motor_task step */
    k_bit_led_enc_tick  = 2, /**< P72 -- D12, toggles per encoder sample */
    k_bit_led_motor_alive = 1, /**< PB1 -- D13, proves motor_task reached loop body */
} status_led_pins_t;

static void leds_init(void)
{
    gpio_set_output(k_port_a, k_bit_led_heartbeat);
    gpio_write(k_port_a, k_bit_led_heartbeat, false);

    gpio_set_output(k_port_b, k_bit_led_init_done);
    gpio_write(k_port_b, k_bit_led_init_done, false);

    gpio_set_output(k_port_7, k_bit_led_motor_run);
    gpio_write(k_port_7, k_bit_led_motor_run, false);

    gpio_set_output(k_port_7, k_bit_led_enc_tick);
    gpio_write(k_port_7, k_bit_led_enc_tick, false);

    gpio_set_output(k_port_b, k_bit_led_motor_alive);
    gpio_write(k_port_b, k_bit_led_motor_alive, false);
}

/*
 ==========================================================================
 * Per-motor wiring (mirrors encoder_test / src/inc/hardware_config.h).
 *
 * | Idx | Wheel | GPTW | IN1  | IN2  | DRVOFF | nSLEEP | Encoder |
 * |-----|-------|------|------|------|--------|--------|---------|
 * |  0  | FL    | 0    | P17  | P23  | P61    | P60    | MTU1    |
 * |  1  | FR    | 1    | PC3  | P22  | P63    | P62    | MTU2    |
 * |  2  | BR    | 2    | P86  | PE3  | PE0    | P64    | TPU1    |
 * |  3  | BL    | 3    | PC6  | PE7  | PE2    | PE1    | TPU2    |
 *
 * direction_sign compensates for the physical wiring: M0 (FL) and M3 (BL)
 * are reversed on this PCB revision, so +50% duty for "forward" applies as
 * -50% to those channels.
 *
 ==========================================================================
 */
typedef struct {
    rx_gptw_channel_t gptw_channel;
```

```c
    rx_port_pin_t      in2_pin;
    rx_port_pin_t      in1_pin;
    uint8_t            drvoff_port;
    uint8_t            drvoff_pin;
    uint8_t            nsleep_port;
    uint8_t            nsleep_pin;
    bool               is_tpu;
    uint8_t            timer_channel;
    int8_t             direction_sign;
} motor_wiring_t;

typedef enum : uint8_t {
    k_motor_count = 4,
} motor_const_t;

static const motor_wiring_t k_motors[k_motor_count] = {
    /* M0 = FL, wired backwards */
    {.gptw_channel = k_gptw_channel_0, .in2_pin = k_rx_p2_3, .in1_pin = k_rx_p1_7,
     .drvoff_port = k_port_6, .drvoff_pin = 1, .nsleep_port = k_port_6, .nsleep_pin = 0,
     .is_tpu = false, .timer_channel = 1, .direction_sign = -1},
    /* M1 = FR */
    {.gptw_channel = k_gptw_channel_1, .in2_pin = k_rx_p2_2, .in1_pin = k_rx_pc_3,
     .drvoff_port = k_port_6, .drvoff_pin = 3, .nsleep_port = k_port_6, .nsleep_pin = 2,
     .is_tpu = false, .timer_channel = 2, .direction_sign = +1},
    /* M2 = BR */
    {.gptw_channel = k_gptw_channel_2, .in2_pin = k_rx_pe_3, .in1_pin = k_rx_p8_6,
     .drvoff_port = k_port_e, .drvoff_pin = 0, .nsleep_port = k_port_6, .nsleep_pin = 4,
     .is_tpu = true,  .timer_channel = 1, .direction_sign = +1},
    /* M3 = BL, wired backwards */
    {.gptw_channel = k_gptw_channel_3, .in2_pin = k_rx_pe_7, .in1_pin = k_rx_pc_6,
     .drvoff_port = k_port_e, .drvoff_pin = 2, .nsleep_port = k_port_e, .nsleep_pin = 1,
     .is_tpu = true,  .timer_channel = 2, .direction_sign = -1},
};

/*
 ========================================================================
 * Bounded busy-wait (~1 ms per unit at 240 MHz ICLK, -O1, volatile loop).
 * Only used in main() before tx_kernel_enter() for the DRV8263 tWAKE window;
 * after that, every delay goes through tx_thread_sleep().
 *
 ========================================================================
 */
typedef enum : uint32_t {
    k_iters_per_ms = 40000U,
} delay_const_t;

static void busy_wait_ms(uint32_t ms)
{
    for (uint32_t m = 0; m < ms; m++) {
        for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
            __asm__ volatile("nop");
        }
    }
}

/*
 ========================================================================
 * DRV8263H power-up: DRVOFF high first, nSLEEP high, 10 ms tWAKE, DRVOFF low.
 *
 ========================================================================
 */
static void motor_drv_gpio_init(void)
{
    for (uint8_t i = 0; i < k_motor_count; i++) {
        gpio_set_output(k_motors[i].drvoff_port, k_motors[i].drvoff_pin);
        gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, true);
    }
    for (uint8_t i = 0; i < k_motor_count; i++) {
        gpio_set_output(k_motors[i].nsleep_port, k_motors[i].nsleep_pin);
        gpio_write(k_motors[i].nsleep_port, k_motors[i].nsleep_pin, true);
    }
    busy_wait_ms(10);
    for (uint8_t i = 0; i < k_motor_count; i++) {
        gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, false);
    }
}

/*
 ========================================================================
 * GPTW PWM init via rx_motor lib (same config as motor_spin_test).
 *
 ========================================================================
 */
typedef enum : uint32_t {
    k_pwm_freq_hz  = 20000U,
    k_deadtime_ns  = 1000U,
} pwm_const_t;
```

```
static bool motor_pwm_init(rx_motor_handle_t handles[k_motor_count])
{
    for (uint8_t i = 0; i < k_motor_count; i++) {
        const rx_motor_config_t cfg = {
            .channel      = k_motors[i].gptw_channel,
            .output_a     = k_gptw_output_a,
            .output_b     = k_gptw_output_b,
            .pwm_freq_hz  = k_pwm_freq_hz,
            .dead_time_ns = k_deadtime_ns,
            .invert_pwm   = false,
            .port_a_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in2_pin),
            .bit_a        = (uint8_t)rx_pin_from_pin(k_motors[i].in2_pin),
            .port_b_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in1_pin),
            .bit_b        = (uint8_t)rx_pin_from_pin(k_motors[i].in1_pin),
        };
        if (rx_motor_init(&handles[i], &cfg) != k_rx_ok) {
            return false;
        }
    }
    return true;
}

/*
=============================================================================
 * Encoder init -- direct-register, bypassing rx_mpc / rx_mtu_start which
 * don't actually write PFS or TSTRA on this HAL revision.  Copied verbatim
 * from the known-good encoder_test/main.c encoders_init() routine.
 *
=============================================================================
*/
#define REG8_AT(a)  (*(volatile uint8_t  *)(uintptr_t)(a))
#define REG16_AT(a) (*(volatile uint16_t *)(uintptr_t)(a))

static bool encoders_init(void)
{
    /* Step 1: release MTU (MSTPA9) + TPU (MSTPA13) module clocks. */
    *prcr_reg() = k_rx_prcr_unlock_all;
    system_regs()->mstpcra &= ~(uint32_t)((1UL « 9) | (uint32_t)k_tpu_mstpcra_mstpa13);
    *prcr_reg() = k_rx_prcr_lock;

    /* Step 2 + 3: MPC pin sequence PMR=0, write PFS, PMR=1 for every
     * encoder phase input. */
    volatile uint8_t *pwpr  = (volatile uint8_t *)0x0008C11FU;
    volatile uint8_t *p2pmr = (volatile uint8_t *)0x0008C062U;
    volatile uint8_t *papmr = (volatile uint8_t *)0x0008C06AU;
    volatile uint8_t *pbpmr = (volatile uint8_t *)0x0008C06BU;
    volatile uint8_t *pcpmr = (volatile uint8_t *)0x0008C06CU;

    *p2pmr &= (uint8_t)~(uint8_t)((1U « 4) | (1U « 5));
    *papmr &= (uint8_t)~(uint8_t)((1U « 1) | (1U « 3));
    *pcpmr &= (uint8_t)~(uint8_t)((1U « 0) | (1U « 2) | (1U « 5));
    *pbpmr &= (uint8_t)~(uint8_t)(1U « 3);

    *pwpr = 0x00U;
    *pwpr = 0x40U;
    REG8_AT(0x0008C140U + 2U * 8U + 4U)  = 0x02U; /* P24 MTCLKA */
    REG8_AT(0x0008C140U + 2U * 8U + 5U)  = 0x02U; /* P25 MTCLKB */
    REG8_AT(0x0008C140U + 10U * 8U + 1U) = 0x02U; /* PA1 MTCLKC */
    REG8_AT(0x0008C140U + 12U * 8U + 5U) = 0x02U; /* PC5 MTCLKD */
    REG8_AT(0x0008C140U + 12U * 8U + 2U) = 0x03U; /* PC2 TCLKA  */
    REG8_AT(0x0008C140U + 10U * 8U + 3U) = 0x04U; /* PA3 TCLKB  */
    REG8_AT(0x0008C140U + 12U * 8U + 0U) = 0x03U; /* PC0 TCLKC  */
    REG8_AT(0x0008C140U + 11U * 8U + 3U) = 0x04U; /* PB3 TCLKD  */
    *pwpr = 0x00U;
    *pwpr = 0x80U;

    *p2pmr |= (uint8_t)((1U « 4) | (1U « 5));
    *papmr |= (uint8_t)((1U « 1) | (1U « 3));
    *pcpmr |= (uint8_t)((1U « 0) | (1U « 2) | (1U « 5));
    *pbpmr |= (uint8_t)(1U « 3);

    /* Phase counting mode 1 on MTU1 + MTU2 */
    volatile rx_mtu_channel_regs_t *m1 = mtu1();
    m1->tcr = 0;  m1->tmdr = 0x04U;
    m1->tiorh = 0; m1->tiorl = 0;
    m1->tier = 0;  m1->tsr = 0;  m1->tcnt = 0;
    volatile rx_mtu_channel_regs_t *m2 = mtu2();
    m2->tcr = 0;  m2->tmdr = 0x04U;
    m2->tiorh = 0; m2->tiorl = 0;
    m2->tier = 0;  m2->tsr = 0;  m2->tcnt = 0;

    /* Phase counting mode 1 on TPU1 + TPU2 */
    volatile rx_tpu_regs_t *t1 = tpu1();
    t1->tcr = 0;  t1->tmdr = 0x04U;  t1->tcnt = 0;
    volatile rx_tpu_regs_t *t2 = tpu2();
    t2->tcr = 0;  t2->tmdr = 0x04U;  t2->tcnt = 0;
```

```

    /* Clear TPU noise filters from any prior-run leftover state */
    tpu_control()->nfcr[1] = 0;
    tpu_control()->nfcr[2] = 0;

    /* Start all four counters */
    mtu_tstra()->tstr   |= (uint8_t)(k_mtu_tstr_cst1 | k_mtu_tstr_cst2);
    tpu_control()->tstr |= (uint8_t)(k_tpu_tstr_cst1 | k_tpu_tstr_cst2);

    return true;
}

static uint16_t encoder_read_raw(uint8_t motor_idx)
{
    /* IMPORTANT -- indices 2 and 3 are INTENTIONALLY SWAPPED relative to the
     * k_motors[] table's `.timer_channel` field and the previous "M2->TPU1,
     * M3->TPU2" comment.  Cross-check against motor_spin_test IPROPI on
     * 2026-04-21 showed M2 (BR) drawing ~0 current at every duty (dead
     * motor, hardware issue) while the encoder at TPU1 was counting fast
     * and the encoder at TPU2 was pinned at 0.  The only consistent reading
     * is that the physical encoder harness wires BR's encoder to TPU2
     * (TCLKC/TCLKD on PC0/PB3) and BL's encoder to TPU1 (TCLKA/TCLKB on
     * PC2/PA3) -- the opposite of what the k_motors[] comment claims.
     * Swapping the indices here makes m2 track BR and m3 track BL so the
     * CSV column matches the motor driver index in motor_spin_test. */
    static const uintptr_t k_tcnt_addr[k_motor_count] = {
        0x000C1386U,       /* MTU1 -- M0 (FL) */
        0x000C1406U,       /* MTU2 -- M1 (FR) */
        0x00088130U + 6U,  /* TPU2 -- M2 (BR): harness wires BR -> TPU2 */
        0x00088120U + 6U,  /* TPU1 -- M3 (BL): harness wires BL -> TPU1 */
    };
    return REG16_AT(k_tcnt_addr[motor_idx]);
}

/*
 ============================================================================
 * Console helpers -- thin wrappers around sci9_debug_* for readability.
 *
 ============================================================================
 */
static void print_s16_dec(int16_t v)
{
    char buf[8];
    uint8_t  i       = 0;
    uint32_t u;
    bool     negative = v < 0;

    u = negative ? (uint32_t)(-(int32_t)v) : (uint32_t)v;
    if (u == 0U) {
        sci9_debug_putc('0');
        return;
    }
    if (negative) {
        sci9_debug_putc('-');
    }
    while (u != 0U) {
        buf[i++] = (char)('0' + (u % 10U));
        u /= 10U;
    }
    while (i-- > 0U) {
        sci9_debug_putc(buf[i]);
    }
}

/*
 ============================================================================
 * Duty sweep state machine -- 21 duty steps from -100 to +100 in 10% steps,
 * reversed at the endpoints.  motor_task advances one step per wakeup.
 *
 ============================================================================
 */
typedef enum : uint8_t {
    k_sweep_step_count = 21U, /**< -100, -90, ..., 0, ..., +100 */
} sweep_const_t;

static const int8_t k_sweep_table[k_sweep_step_count] = {
    -100, -90, -80, -70, -60, -50, -40, -30, -20, -10,
       0,
     +10, +20, +30, +40, +50, +60, +70, +80, +90, +100,
};

/*
 ============================================================================
 * ThreadX objects
 *
 ============================================================================
 */
```

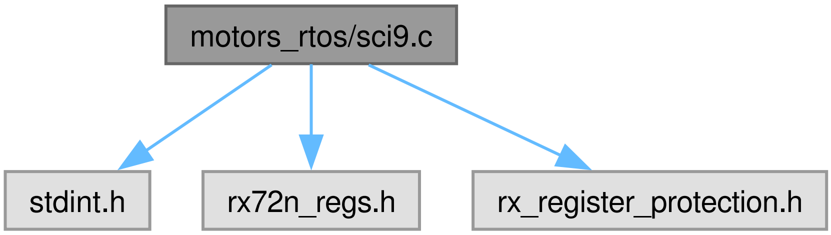
```c
typedef enum : uint16_t {
    k_motor_task_stack_sz   = 2048U,
    k_encoder_task_stack_sz = 2048U,
} rtos_stack_t;

typedef enum : uint8_t {
    k_encoder_task_priority = 9U,  /**< higher priority: prints CSV, keeps console responsive */
    k_motor_task_priority   = 10U, /**< lower priority: sweeps duty, preempted by encoder */
    k_motor_sleep_ticks     = 50U, /**< 50 * 10 ms = 500 ms */
    k_encoder_sleep_ticks   = 10U, /**< 10 * 10 ms = 100 ms */
} rtos_prio_t;

static TX_THREAD s_motor_tcb;
static TX_THREAD s_encoder_tcb;
static uint8_t   s_motor_stack[k_motor_task_stack_sz];
static uint8_t   s_encoder_stack[k_encoder_task_stack_sz];

/* Motor handles initialised in main() before the scheduler starts. */
static rx_motor_handle_t s_motors[k_motor_count];

/* Most recent motor duty -- written by motor_task, read by encoder_task for
 * CSV logging.  Signed volatile byte is atomic on RX so no mutex needed. */
static volatile int8_t s_current_duty = 0;

/* Diagnostic counters -- help identify *where* motor_task is making progress.
 * All three are volatile so the compiler can't optimise them away. */
static volatile uint32_t g_motor_task_entries     = 0U; /**< bumped once on task entry */
static volatile uint32_t g_motor_task_loops       = 0U; /**< bumped each iteration BEFORE set_duty */
static volatile uint32_t g_motor_task_loops_after = 0U; /**< bumped each iteration AFTER  set_duty */
static volatile uint32_t g_motor_create_status    = 0xFFFFFFFFU; /**< tx_thread_create return for motor_task */
static volatile uint32_t g_encoder_create_status  = 0xFFFFFFFFU; /**< tx_thread_create return for encoder_task */

extern volatile uint32_t g_cmt0_isr_count; /**< from cmt0.c -- verifies CMT0 tick ISR is firing */

/* ThreadX kernel internals -- used by Option 1/1b diagnostic in encoder_task.
 * tx_thread_sleep.c:89-118 has four TX_CALLER_ERROR (0x13) return paths:
 *   :89   thread_ptr == NULL          -> observe via cur
 *   :99   system_state != 0           -> observe via ss  (ISR wrapping issue)
 *   :111  thread_ptr == timer thread  -> observe via cur
 *   :132  preempt_disable != 0        -> observe via pd  (already ruled out: pd=0) */
extern UINT       _tx_thread_preempt_disable;
extern ULONG      _tx_thread_system_state;
extern TX_THREAD *_tx_thread_current_ptr;

/*
  ==============================================================================
 * motor_task -- step the duty sweep across all 4 motors every 500 ms.
 *
  ==============================================================================
  */
static void motor_task_entry(ULONG arg)
{
    (void)arg;

    g_motor_task_entries = 1U;
    gpio_write(k_port_b, k_bit_led_motor_alive, true);

    /* Explicit 0 duty on entry for a clean baseline before the sweep. */
    for (uint8_t i = 0; i < k_motor_count; i++) {
        (void)rx_motor_set_duty(&s_motors[i], 0.0F);
    }

    uint8_t step = 0U;

    for (;;) {
        gpio_toggle(k_port_7, k_bit_led_motor_run);

        const int8_t duty = k_sweep_table[step];
        s_current_duty = duty;

        for (uint8_t i = 0; i < k_motor_count; i++) {
            const float signed_duty = (float)(int16_t)duty *
                                      (float)(int16_t)k_motors[i].direction_sign;
            (void)rx_motor_set_duty(&s_motors[i], signed_duty);
        }

        step = (uint8_t)((step + 1U) % k_sweep_step_count);

        (void)tx_thread_sleep((ULONG)k_motor_sleep_ticks);
    }
}

/*
  ==============================================================================
 * encoder_task -- 10 Hz CSV dump + heartbeat LED.
 *
 * CSV header (printed once at task start): t_ms,duty,m0,m1,m2,m3
```

```
 * Each subsequent line: uptime,duty,cnt0,cnt1,cnt2,cnt3
 * ===========================================================================
 */
static void encoder_task_entry(ULONG arg)
{
    (void)arg;

    sci9_debug_puts("t_ms,cmt0_cnt,mot_state,m_ent,duty,m0,m1,m2,m3\n");

    uint32_t t_ms = 0U;

    for (;;) {
        gpio_toggle(k_port_a, k_bit_led_heartbeat);
        gpio_toggle(k_port_7, k_bit_led_enc_tick);

        const uint16_t c0 = encoder_read_raw(0U);
        const uint16_t c1 = encoder_read_raw(1U);
        const uint16_t c2 = encoder_read_raw(2U);
        const uint16_t c3 = encoder_read_raw(3U);


        print_s16_dec((int16_t)(t_ms & 0x7FFFU));
        sci9_debug_putc(',');
        print_s16_dec((int16_t)(g_cmt0_isr_count & 0x7FFFU));
        sci9_debug_putc(',');
        print_s16_dec((int16_t)s_motor_tcb.tx_thread_state);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)g_motor_task_entries);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)s_current_duty);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)c0);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)c1);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)c2);
        sci9_debug_putc(',');
        print_s16_dec((int16_t)c3);
        sci9_debug_putc('\n');

        /* Option 1b diagnostic: capture all TX_CALLER_ERROR discriminators
         * BEFORE sleep plus its return value AFTER.  ss and cur distinguish
         * the three remaining TX_CALLER_ERROR paths in tx_thread_sleep.c. */
        const UINT        pd  = _tx_thread_preempt_disable;
        const ULONG       ss  = _tx_thread_system_state;
        const TX_THREAD  *cur = _tx_thread_current_ptr;
        const UINT        ret = tx_thread_sleep((ULONG)k_encoder_sleep_ticks);
        sci9_debug_puts("pd=");
        print_s16_dec((int16_t)pd);
        sci9_debug_puts(" ss=");
        sci9_debug_puthex32((uint32_t)ss);
        sci9_debug_puts(" cur=");
        sci9_debug_puthex32((uint32_t)(uintptr_t)cur);
        sci9_debug_puts(" ret=");
        sci9_debug_puthex16((uint16_t)ret);
        sci9_debug_putc('\n');
        t_ms += 100U;
    }
}

/*
 * ===========================================================================
 * ThreadX application-define -- create both tasks (hardware already init'd).
 * ===========================================================================
 */
void tx_application_define(void *first_unused_memory)
{
    (void)first_unused_memory;

    g_encoder_create_status = (uint32_t)tx_thread_create(
        &s_encoder_tcb, "encoder", encoder_task_entry, 0U,
        s_encoder_stack, (ULONG)k_encoder_task_stack_sz,
        (UINT)k_encoder_task_priority,
        (UINT)k_encoder_task_priority,
        TX_NO_TIME_SLICE, TX_AUTO_START);

    g_motor_create_status = (uint32_t)tx_thread_create(
        &s_motor_tcb, "motor", motor_task_entry, 0U,
        s_motor_stack, (ULONG)k_motor_task_stack_sz,
        (UINT)k_motor_task_priority,
        (UINT)k_motor_task_priority,
        TX_NO_TIME_SLICE, TX_AUTO_START);
}

/*
```

```
=============================================================================
 * Fatal error trap -- park all four motors and halt with the FAIL LED lit.
 *
=============================================================================
*/
static void error_park(const char *msg)
{
    sci9_debug_puts("FATAL ");
    sci9_debug_puts(msg);
    sci9_debug_puts("\n");

    /* Best-effort motor safe: attempt a 0-duty write (may fail if rx_motor
     * isn't initialised, but that's OK since GPTW would be idle in that case). */
    for (uint8_t i = 0; i < k_motor_count; i++) {
        (void)rx_motor_set_duty(&s_motors[i], 0.0F);
    }

    for (;;) {
        __asm__ volatile("nop");
    }
}

/*
=============================================================================
 * main() -- runs ONCE on reset, init everything, hand off to ThreadX forever.
 *
=============================================================================
*/
int main(void)
{
    clock_init();
    leds_init();
    sci9_debug_init();

    sci9_debug_puts("\n\n");
    sci9_debug_puts("motors_rtos: ThreadX + rx_motor + rx_encoder bring-up\n");
    sci9_debug_puts("clock=240MHz pclkb=60MHz tick=100Hz\n");

    motor_drv_gpio_init();
    sci9_debug_puts("drv8263 armed\n");

    if (!motor_pwm_init(s_motors)) {
        error_park("motor_pwm_init");
    }
    sci9_debug_puts("motors 0..3 init ok\n");

    if (!encoders_init()) {
        error_park("encoders_init");
    }
    sci9_debug_puts("encoders 0..3 init ok\n");

    gpio_write(k_port_b, k_bit_led_init_done, true);
    sci9_debug_puts("entering scheduler\n");

    cmt0_init();
    tx_kernel_enter();

    /* tx_kernel_enter() never returns; fall-through trap if it somehow does. */
    for (;;) {
        __asm__ volatile("nop");
    }
    return 0;
}
```

4.9 motors_rtos/sci9.c File Reference

#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
Include dependency graph for sci9.c:



Enumerations

- enum sci9_addrs_t : uintptr_t { k_sci9_base = 0x000D0020U }
- enum sci9_cfg_t : uint8_t {
  k_psel_sci9 = 0x0AU , k_pb6_mask = (uint8_t)(1U << 6U) , k_pb7_mask = (uint8_t)(1U << 7U) , k_mstpc_sci9_bit = 26U ,
  k_pwpr_unlock_b0 = 0x00U , k_pwpr_pfswe = 0x40U , k_pwpr_b0wi = 0x80U , k_brr_115200 = 31U ,
  k_smr_8n1_async = 0x00U , k_scr_te_only = 0x20U , k_scr_txrx_enabled = 0x30U , k_scr_disabled = 0x00U ,
  k_ssr_tdre = 0x80U , k_scmr_default = 0xF2U , k_semr_default = 0x00U , k_hex_nibble_mask = 0x0FU }
- enum sci9_settle_t : uint16_t { k_brr_settle_loops = 1000U }
- enum hex_shifts_t : uint8_t { k_hex_shift_init32 = 28U , k_hex_shift_init16 = 12U , k_hex_shift_step = 4U }

Functions

- static volatile uint8_t * sci9_smr (void)
- static volatile uint8_t * sci9_brr (void)
- static volatile uint8_t * sci9_scr (void)
- static volatile uint8_t * sci9_tdr (void)
- static volatile uint8_t * sci9_ssr (void)
- static volatile uint8_t * sci9_scmr (void)
- static volatile uint8_t * sci9_semr (void)
- void sci9_debug_init (void)
- void sci9_debug_putc (char c)
- void sci9_debug_puts (const char *s)
- void sci9_debug_puthex32 (uint32_t v)
- void sci9_debug_puthex16 (uint16_t v)

4.9.1 Enumeration Type Documentation

4.9.1.1 hex_shifts_t

enum hex_shifts_t : uint8_t

Enumerator

k_hex_shift_init32	
k_hex_shift_init16	
k_hex_shift_step	

Definition at line 62 of file sci9.c.

```
00062              : uint8_t {
00063     k_hex_shift_init32 = 28U,
00064     k_hex_shift_init16 = 12U,
00065     k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
```

4.9.1.2 sci9_addrs_t

enum sci9_addrs_t : uintptr_t

Enumerator

k_sci9_base	

Definition at line 35 of file sci9.c.

```
00035              : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
```

4.9.1.3 sci9_cfg_t

enum sci9_cfg_t : uint8_t

Enumerator

k_psel_sci9	SCI9 RXD/TXD function
k_pb6_mask	
k_pb7_mask	
k_mstpc_sci9_bit	
k_pwpr_unlock_b0	
k_pwpr_pfswe	
k_pwpr_b0wi	
k_brr_115200	
k_smr_8n1_async	
k_scr_te_only	CKE=00, MPIE=0, RIE=0, TE=1, RE=0
k_scr_txrx_enabled	CKE=00, MPIE=0, RIE=0, TE=1, RE=1
k_scr_disabled	
k_ssr_tdre	
k_scmr_default	
k_semr_default	
k_hex_nibble_mask	

Definition at line 39 of file sci9.c.

```
00039              : uint8_t {
00040     k_psel_sci9        = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask         = (uint8_t)(1U « 6U),
00042     k_pb7_mask         = (uint8_t)(1U « 7U),
00043     k_mstpc_sci9_bit   = 26U, /* MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0   = 0x00U,
00045     k_pwpr_pfswe       = 0x40U,
00046     k_pwpr_b0wi        = 0x80U,
00047     k_brr_115200       = 31U, /* SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117
        kbaud (1.7% over 115200) */
00048     k_smr_8n1_async    = 0x00U,
00049     k_scr_te_only      = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled     = 0x00U,
00052     k_ssr_tdre         = 0x80U,
00053     k_scmr_default     = 0xF2U,
00054     k_semr_default     = 0x00U,
00055     k_hex_nibble_mask  = 0x0FU,
00056 } sci9_cfg_t;
```

4.9.1.4 sci9_settle_t

enum sci9_settle_t : uint16_t

Enumerator

k_brr_settle_loops	Match production uart.c (∼8.68 us @ 115200)

Definition at line 58 of file sci9.c.

```
00058             : uint16_t {
00059     k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
```

4.9.2 Function Documentation

4.9.2.1 sci9_brr()

volatile uint8_t * sci9_brr (
    void) [inline], [static]

Definition at line 76 of file sci9.c.

```
00077 {
00078     return (volatile uint8_t *)(k_sci9_base + 1U);
00079 }
```

References k_sci9_base.

Referenced by sci9_debug_init().

Here is the caller graph for this function:

4.9.2.2 sci9_debug_init()

void sci9_debug_init (
    void)

Definition at line 104 of file sci9.c.

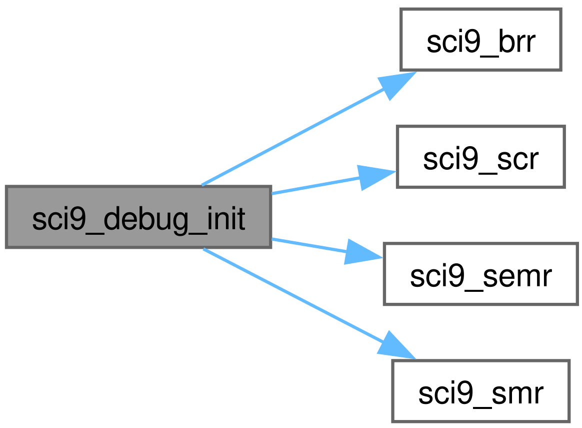
```
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL << (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */
00121
```

```
    /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
     * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
    portb()->pdr |= (uint8_t)k_pb7_mask;
    portb()->pdr &= (uint8_t) ~k_pb6_mask;
    portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);

    /* Step 4: SCI9 init.  Match the production order:
     *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
     * Don't touch SCMR (reset default 0xF2 is correct). */
    *sci9_scr() = k_scr_disabled;
    *sci9_smr() = k_smr_8n1_async;
    *sci9_brr() = k_brr_115200;

    /* Wait roughly one bit time before enabling TX/RX. */
    for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
        __asm__ volatile("nop");
    }

    *sci9_scr()  = k_scr_txrx_enabled;
    *sci9_semr() = k_semr_default;
}
```

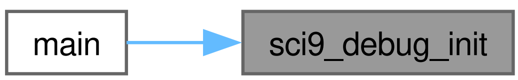
References k_brr_115200, k_brr_settle_loops, k_mstpc_sci9_bit, k_pb6_mask, k_pb7_mask, k_psel_sci9, k_pwpr_b0wi, k_pwpr_pfswe, k_pwpr_unlock_b0, k_scr_disabled, k_scr_txrx_enabled, k_semr_default, k_smr_8n1_async, sci9_brr(), sci9_scr(), sci9_semr(), and sci9_smr().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.2.3 sci9_debug_putc()

```
void sci9_debug_putc (
        char c)
```

Definition at line 147 of file sci9.c.

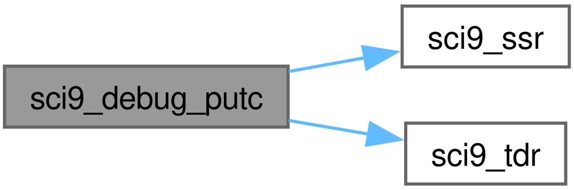
```
{
    while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
        /* spin */
    }
    *sci9_tdr() = (uint8_t)c;
```

```
}
```

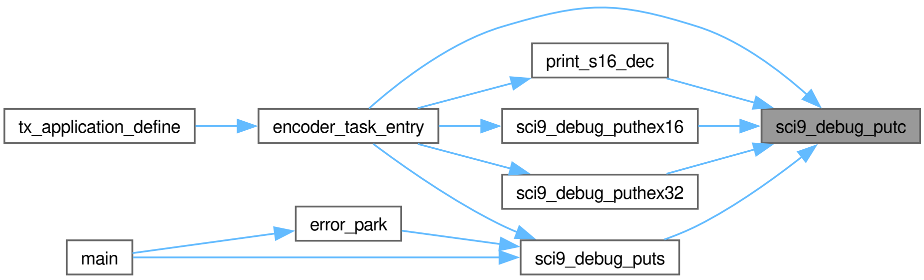
References k_ssr_tdre, sci9_ssr(), and sci9_tdr().

Referenced by encoder_task_entry(), print_s16_dec(), sci9_debug_puthex16(), sci9_debug_puthex32(), and sci9_debug_puts().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.2.4 sci9_debug_puthex16()

```
void sci9_debug_puthex16 (
        uint16_t v)
```

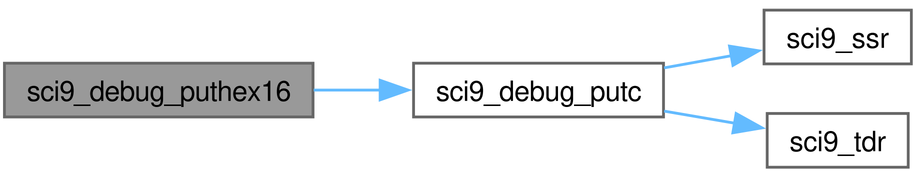
Definition at line 179 of file sci9.c.

```
{
    static const char hex_digits[] = "0123456789ABCDEF";
    sci9_debug_putc('0');
    sci9_debug_putc('x');
    for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
         shift -= (int8_t)k_hex_shift_step) {
        sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
    }
}
```

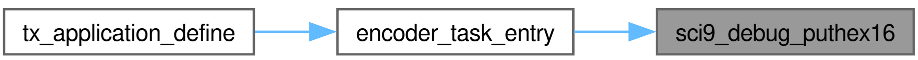
References k_hex_nibble_mask, k_hex_shift_init16, k_hex_shift_step, and sci9_debug_putc().

Referenced by encoder_task_entry().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.2.5 sci9_debug_puthex32()

void sci9_debug_puthex32 (
uint32_t v)

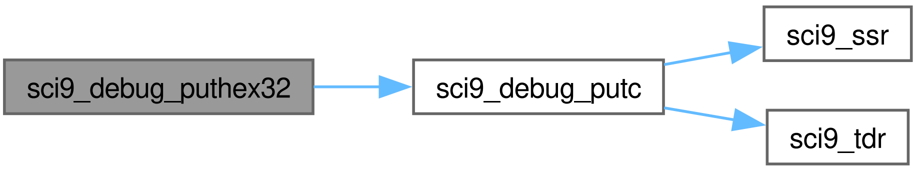
Definition at line 168 of file sci9.c.

```
{
    static const char hex_digits[] = "0123456789ABCDEF";
    sci9_debug_putc('0');
    sci9_debug_putc('x');
    for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
         shift -= (int8_t)k_hex_shift_step) {
        sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
    }
}
```

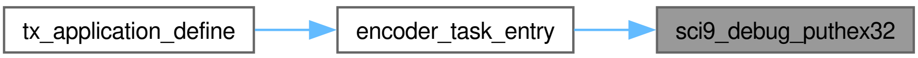
References k_hex_nibble_mask, k_hex_shift_init32, k_hex_shift_step, and sci9_debug_putc().

Referenced by encoder_task_entry().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.2.6 sci9_debug_puts()

void sci9_debug_puts (
const char * s)

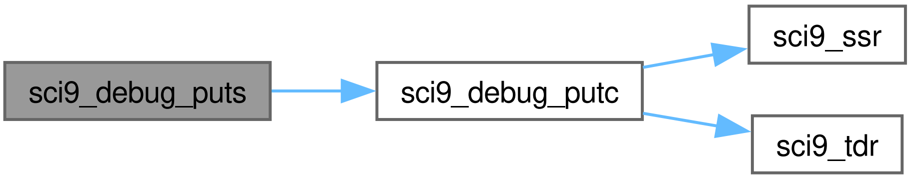
Definition at line 155 of file sci9.c.

```
{
    if (s == (const char *)0) {
        return;
    }
    while (*s != '\0') {
        if (*s == '\n') {
            sci9_debug_putc('\r');
        }
        sci9_debug_putc(*s++);
    }
}
```

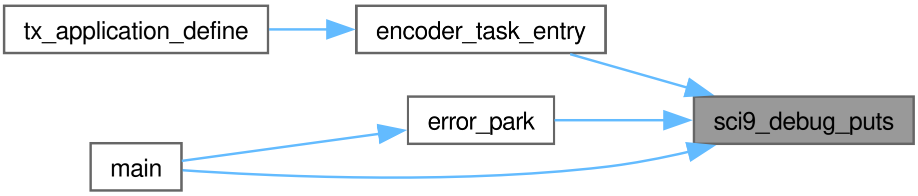
References sci9_debug_putc().

Referenced by encoder_task_entry(), error_park(), and main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.2.7 sci9_scmr()

volatile uint8_t * sci9_scmr (
void) [inline], [static]

Definition at line 92 of file sci9.c.

```
00093 {
00094     return (volatile uint8_t *)(k_sci9_base + 6U);
00095 }
```

References k_sci9_base.

4.9.2.8 sci9_scr()

volatile uint8_t * sci9_scr (
void) [inline], [static]

Definition at line 80 of file sci9.c.

```
00081 {
00082     return (volatile uint8_t *)(k_sci9_base + 2U);
00083 }
```

References k_sci9_base.

Referenced by sci9_debug_init().

Here is the caller graph for this function:

4.9.2.9 sci9_semr()

volatile uint8_t * sci9_semr (
void) [inline], [static]

Definition at line 96 of file sci9.c.

```
00097 {
00098     return (volatile uint8_t *)(k_sci9_base + 7U);
00099 }
```

References k_sci9_base.

Referenced by sci9_debug_init().

Here is the caller graph for this function:

4.9.2.10 sci9_smr()

volatile uint8_t * sci9_smr (
void) [inline], [static]

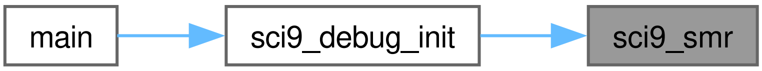
Definition at line 72 of file sci9.c.

```
00073 {
00074     return (volatile uint8_t *)(k_sci9_base + 0U);
00075 }
```

References k_sci9_base.

Referenced by sci9_debug_init().

Here is the caller graph for this function:



4.9.2.11 sci9_ssr()

volatile uint8_t * sci9_ssr (
void) [inline], [static]

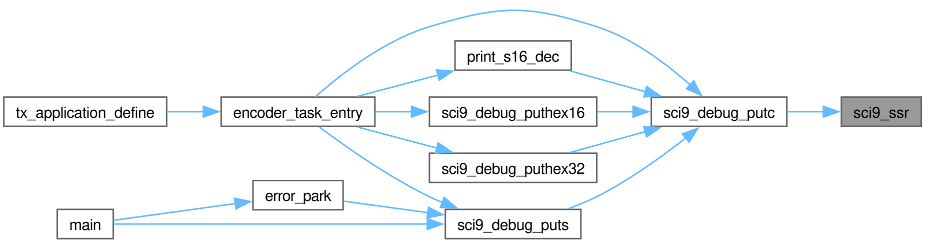
Definition at line 88 of file sci9.c.

```
00089 {
00090     return (volatile uint8_t *)(k_sci9_base + 4U);
00091 }
```

References k_sci9_base.

Referenced by sci9_debug_putc().

Here is the caller graph for this function:



4.9.2.12 sci9_tdr()

volatile uint8_t ∗ sci9_tdr (
void) [inline], [static]

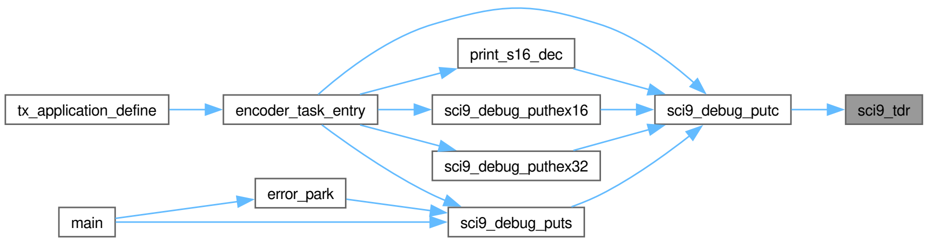
Definition at line 84 of file sci9.c.

```
{
    return (volatile uint8_t *)(k_sci9_base + 3U);
}
```

References k_sci9_base.

Referenced by sci9_debug_putc().

Here is the caller graph for this function:



4.10 sci9.c

Go to the documentation of this file.

```
/**
 * @file sci9_debug.c
 * @brief Minimal polled-mode SCI9 (TXD9 = PB7, RXD9 = PB6) debug UART for usb_test.
 *
 * @details
 * The STAR board routes SCI9 to the on-board CY7C65213 USB-to-UART bridge
 * which appears as /dev/ttyACM0 on the host (or COMx on Windows).  This
 * file provides just enough init + putc/puts to emit ASCII status from
 * the firmware so we can debug the USB CDC bring-up without needing the
 * USB device to actually enumerate.
 *
 * Uses the rx_hal struct accessors (mpc(), portb(), system_regs(), prcr_reg())
 * so the addresses are all compiler-verified via static_assert.
 *
 * Baud = 115200. SCI9 is in the SCI7-11 group that uses PCLKA (120 MHz),
 * not PCLKB. At CKS=0, ABCS=0:
 *   BRR = (PCLKA / (32 * baud)) - 1 = 120000000 / 3686400 - 1 = 31.55 -> 31
 *   Actual baud = PCLKA / (32 * (BRR+1)) = 117187 Hz (+1.7% vs 115200, in tolerance).
 *
 * SPDX-License-Identifier: MIT
 * @copyright Copyright (c) 2026 Locked Inc.
 */

#include <stdint.h>

#include "rx72n_regs.h"
#include "rx_register_protection.h"

/* ==========================================================================
 * SCI9 register access via raw addresses (rx72n_sci_regs.h doesn't expose
 * the per-channel struct as cleanly as we'd like for a quick polled driver).
 * The base 0x000D0020 is taken from the verified header.
 * Register offsets per RX72N HW manual: SMR=0, BRR=1, SCR=2, TDR=3, SSR=4.
 * ========================================================================== */
typedef enum : uintptr_t {
    k_sci9_base = 0x000D0020U,
} sci9_addrs_t;

```

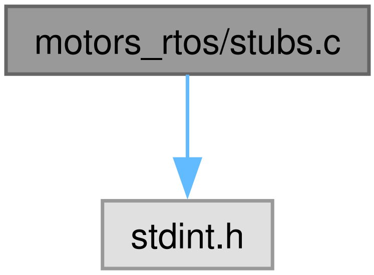
```
typedef enum : uint8_t {
    k_psel_sci9       = 0x0AU, /**< SCI9 RXD/TXD function */
    k_pb6_mask        = (uint8_t)(1U « 6U),
    k_pb7_mask        = (uint8_t)(1U « 7U),
    k_mstpc_sci9_bit  = 26U, /* MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
    k_pwpr_unlock_b0  = 0x00U,
    k_pwpr_pfswe      = 0x40U,
    k_pwpr_b0wi       = 0x80U,
    k_brr_115200      = 31U, /* SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117 kbaud (1.7% over 115200) */
    k_smr_8n1_async   = 0x00U,
    k_scr_te_only     = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
    k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
    k_scr_disabled    = 0x00U,
    k_ssr_tdre        = 0x80U,
    k_scmr_default    = 0xF2U,
    k_semr_default    = 0x00U,
    k_hex_nibble_mask = 0x0FU,
} sci9_cfg_t;

typedef enum : uint16_t {
    k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
} sci9_settle_t;

typedef enum : uint8_t {
    k_hex_shift_init32 = 28U,
    k_hex_shift_init16 = 12U,
    k_hex_shift_step   = 4U,
} hex_shifts_t;

/* ============================================================================
 * SCI9 register accessors (raw addresses since the header doesn't give us a
 * per-channel struct for the SCIi/SCIj quirks).
 * ============================================================================ */
static inline volatile uint8_t *sci9_smr(void)
{
    return (volatile uint8_t *)(k_sci9_base + 0U);
}
static inline volatile uint8_t *sci9_brr(void)
{
    return (volatile uint8_t *)(k_sci9_base + 1U);
}
static inline volatile uint8_t *sci9_scr(void)
{
    return (volatile uint8_t *)(k_sci9_base + 2U);
}
static inline volatile uint8_t *sci9_tdr(void)
{
    return (volatile uint8_t *)(k_sci9_base + 3U);
}
static inline volatile uint8_t *sci9_ssr(void)
{
    return (volatile uint8_t *)(k_sci9_base + 4U);
}
static inline volatile uint8_t *sci9_scmr(void)
{
    return (volatile uint8_t *)(k_sci9_base + 6U);
}
static inline volatile uint8_t *sci9_semr(void)
{
    return (volatile uint8_t *)(k_sci9_base + 7U);
}

/* ============================================================================
 * Initialise SCI9 for 115200 8N1 polled output on PB7 (TXD9).
 * ============================================================================ */
void sci9_debug_init(void)
{
    /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
     * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
    *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
    system_regs()->mstpcrc &= ~(uint32_t)(1UL « (uint32_t)k_mstpc_sci9_bit);
    *prcr_reg() = k_rx_prcr_lock;

    /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
     * accessor so the compiler computes the correct PFS offsets via
     * static_assert-checked layout. */
    mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
    mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
    mpc()->pb6pfs = k_psel_sci9;
    mpc()->pb7pfs = k_psel_sci9;
```

```
    mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
    mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */

    /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
     * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
    portb()->pdr |= (uint8_t)k_pb7_mask;
    portb()->pdr &= (uint8_t) ~k_pb6_mask;
    portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);

    /* Step 4: SCI9 init.  Match the production order:
     *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
     * Don't touch SCMR (reset default 0xF2 is correct). */
    *sci9_scr() = k_scr_disabled;
    *sci9_smr() = k_smr_8n1_async;
    *sci9_brr() = k_brr_115200;

    /* Wait roughly one bit time before enabling TX/RX. */
    for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
        __asm__ volatile("nop");
    }

    *sci9_scr()  = k_scr_txrx_enabled;
    *sci9_semr() = k_semr_default;
}

/*
 ==========================================================================
 * Polled write: spin until TDRE then drop byte into TDR.
 *
 ==========================================================================
 */
void sci9_debug_putc(char c)
{
    while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
        /* spin */
    }
    *sci9_tdr() = (uint8_t)c;
}

void sci9_debug_puts(const char *s)
{
    if (s == (const char *)0) {
        return;
    }
    while (*s != '\0') {
        if (*s == '\n') {
            sci9_debug_putc('\r');
        }
        sci9_debug_putc(*s++);
    }
}

void sci9_debug_puthex32(uint32_t v)
{
    static const char hex_digits[] = "0123456789ABCDEF";
    sci9_debug_putc('0');
    sci9_debug_putc('x');
    for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
         shift -= (int8_t)k_hex_shift_step) {
        sci9_debug_putc(hex_digits[(v >> (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
    }
}

void sci9_debug_puthex16(uint16_t v)
{
    static const char hex_digits[] = "0123456789ABCDEF";
    sci9_debug_putc('0');
    sci9_debug_putc('x');
    for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
         shift -= (int8_t)k_hex_shift_step) {
        sci9_debug_putc(hex_digits[(v >> (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
    }
}
```

4.11 motors_rtos/stubs.c File Reference

No-op log/UART stubs for the motor spin test.

#include <stdint.h>
Include dependency graph for stubs.c:



Functions

- void rx_log_uart_putc (char c)
- void rx_log_uart_puts (const char ∗s)
- void rx_log_uart_putint (int32_t v)
- void rx_log_uart_putuint (uint32_t v)
- void rx_log_uart_puthex (uint32_t v, uint8_t d)
- void uart_debug_putc (char c)
- void uart_debug_puts (const char ∗s)
- void uart_debug_putint (int32_t v)
- void uart_debug_putuint (uint32_t v)
- void uart_debug_puthex (uint32_t v, uint8_t d)

4.11.1 Detailed Description

No-op log/UART stubs for the motor spin test.

The production rx_motor / rx_gptw / rx_mpc libraries call rx_log_error() and RX_ASSERT() macros that ultimately reduce to a handful of UART log primitives. This bench test does not bring up SCI9 or the framed UART ring buffer, so we satisfy the linker with no-op stubs. RX_ASSERT failures still halt (the static-inline internal_rx_fatal_error in rx_check.h disables interrupts and spins).

SPDX-License-Identifier: MIT

Copyright

Definition in file stubs.c.

4.11.2 Function Documentation

4.11.2.1 rx_log_uart_putc()

```
void rx_log_uart_putc (
                char c)
```

Definition at line 20 of file stubs.c.

```
00020 { (void)c; }
```

4.11.2.2 rx_log_uart_puthex()

```
void rx_log_uart_puthex (
        uint32_t v,
        uint8_t d)
```

Definition at line 24 of file stubs.c.

```
00024 { (void)v; (void)d; }
```

4.11.2.3 rx_log_uart_putint()

```
void rx_log_uart_putint (
        int32_t v)
```

Definition at line 22 of file stubs.c.

```
00022 { (void)v; }
```

4.11.2.4 rx_log_uart_puts()

```
void rx_log_uart_puts (
        const char * s)
```

Definition at line 21 of file stubs.c.

```
00021 { (void)s; }
```

4.11.2.5 rx_log_uart_putuint()

```
void rx_log_uart_putuint (
        uint32_t v)
```

Definition at line 23 of file stubs.c.

```
00023 { (void)v; }
```

4.11.2.6 uart_debug_putc()

```
void uart_debug_putc (
        char c)
```

Definition at line 27 of file stubs.c.

```
00027 { (void)c; }
```

4.11.2.7 uart_debug_puthex()

```
void uart_debug_puthex (
        uint32_t v,
        uint8_t d)
```

Definition at line 31 of file stubs.c.

```
00031 { (void)v; (void)d; }
```

4.11.2.8 uart_debug_putint()

```
void uart_debug_putint (
        int32_t v)
```

Definition at line 29 of file stubs.c.

```
00029 { (void)v; }
```

4.11.2.9 uart_debug_puts()

```
void uart_debug_puts (
        const char * s)
```

Definition at line 28 of file stubs.c.

```
00028 { (void)s; }
```

4.11.2.10 uart_debug_putuint()

```
void uart_debug_putuint (
        uint32_t v)
```

Definition at line 30 of file stubs.c.

```
00030 { (void)v; }
```

4.12 stubs.c

Go to the documentation of this file.

```
00001 /**
00002  * @file stubs.c
00003  * @brief No-op log/UART stubs for the motor spin test.
00004  *
00005  * @details
00006  * The production rx_motor / rx_gptw / rx_mpc libraries call rx_log_error()
00007  * and RX_ASSERT() macros that ultimately reduce to a handful of UART log
00008  * primitives. This bench test does not bring up SCI9 or the framed UART
00009  * ring buffer, so we satisfy the linker with no-op stubs. RX_ASSERT
00010  * failures still halt (the static-inline internal_rx_fatal_error in
00011  * rx_check.h disables interrupts and spins).
00012  *
00013  * SPDX-License-Identifier: MIT
00014  * @copyright Copyright (c) 2026 Locked Inc.
00015  */
00016
00017 #include <stdint.h>
00018
00019 /* rx_log_uart family (selected when RX_IS_SIMULATOR == 0). */
00020 void rx_log_uart_putc(char c)              { (void)c; }
00021 void rx_log_uart_puts(const char *s)       { (void)s; }
00022 void rx_log_uart_putint(int32_t v)         { (void)v; }
00023 void rx_log_uart_putuint(uint32_t v)       { (void)v; }
00024 void rx_log_uart_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
00025
00026 /* uart_debug family (called unconditionally by internal_rx_fatal_error). */
00027 void uart_debug_putc(char c)               { (void)c; }
00028 void uart_debug_puts(const char *s)        { (void)s; }
00029 void uart_debug_putint(int32_t v)          { (void)v; }
00030 void uart_debug_putuint(uint32_t v)        { (void)v; }
00031 void uart_debug_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
```